Supplementary Material: When the Interval Is Smaller Than the Instrument

GUSTAVO PEDRO RICOU, School of Computer Science and Statistics, Trinity College Dublin, Ireland

This document is supplementary material and is not part of the main submission. Main-text references of the form "supplementary material S*n*" resolve here. The provenance of every section — what moved here, from where, and when — is indexed in the artefact at `docs/supplement_index.md`.

Several sections below refer to "internal review" rounds and to material moved between revisions. These were adversarial reviews conducted within the project before submission, against the standards of the venues considered; the manuscript has not been submitted to, or reviewed by, any journal. They are recorded because the corrections they produced are part of the evidence, and because an audit whose own failures are hidden is worth less than one whose are not.

1 S1: The replay-rate provenance gap, in full

Auditing the corpus turned the same scrutiny on our own record-keeping, and it did not survive it either. The main text summarises this episode; this section is the full record, kept because the protocol rule it produced is weaker without it.

Replay plans built from StatsBomb carry a **baked-in** 120× **time compression**: a plan's emission offset is already the match time divided by 120. The producer's `--speedup` flag then multiplies that. So `--speedup 1` against such a plan is *not* real time, it is 120×; true real time needs $1/120$. The same flag means different things depending on which plan it is pointed at, and getting it wrong is silent: the run completes, the wall time looks unremarkable, and the numbers stay plausible. We lost a campaign to it before adding a pre-flight that derives the value from the plan and then checks the achieved rate against elapsed wall time.

The part we cannot repair is retrospective. **No surviving artefact records the achieved replay rate of any run we report.** Several superseded Testbed A scenarios record the nominal `--speedup` *flag*, which is not the same thing: converting a flag to a rate requires the plan's compression, and every plan those files name has since been deleted. Everywhere else the orchestrator wrote `speedup` into a per-campaign summary that lived beside the raw per-run data, and for the earliest corpora only the aggregated CSVs were kept. For the E1 corpus of the main text's powered-transport section the surviving evidence is contradictory: the commit that introduced it states the runs were made at true real time, the campaign script reconstructed afterwards specifies `--speedup 10`, and the flag's meaning was corrected 21 hours later the same day. E1 was replayed at 1×, 120× or 1200×, and we cannot say which.

The rate is recoverable from the measurements, if not from the metadata. Having argued all paper that a physical constraint can decide what record-keeping cannot, we applied the same move here, and it works. The start-up cost of the main text's attribution section is a clock: it lasts a fixed 102.6 ms of *wall* time, so the number of events it swallows depends directly on the replay rate. Counting how many events per run carry it therefore reads the rate back out of the data.

The E1 runs sort into cells by how many events they matched, and one cell is diagnostic. Fifteen Kafka runs matched exactly eight events, and their median scheduling lag is **52.34** ms (range 51.60–53.01) — not 103 ms, and not 1.6 ms. A median of eight values is the mean of the fourth and fifth. The only way to land midway between the two modes is for exactly four of the eight to be late, so the fourth value is a fast event and the fifth a slow one: $(1.59 + 103.5)/2 = 52.6$ ms

Author's Contact Information: Gustavo Pedro Ricou, School of Computer Science and Statistics, Trinity College Dublin, Dublin, Ireland, pedrorig@tcd.ie.

against 52.34 observed. Four is also exactly what the loop trace shows at a verified real-time rate (the window table in the main text), and it is what the plan predicts, since five events share t_{sim}=0 and the first of them pays the block rather than inheriting it.

That cell excludes the accelerated alternatives outright. At 120× the prologue spans 12.3 s of match time and holds 16 events; at 1200×, 123 s and 112 events. Under either, all eight matched events would be inside it and the cell would read 103 ms. It reads half that. **E1 was replayed at true real time**, the commit record was right, and the campaign script reconstructed afterwards describes a different, earlier campaign. The remaining cells agree: runs matching five or seven events read 103 ms, as four late events out of five or seven must give.

The inference has since been checked against the original script, and it holds. Archiving the cloud driver before releasing it turned up the campaign as actually invoked, in a shell script in the operator's home directory that no part of the repository had ever referenced. It specifies `--speedup 0.008333` — `1/120`, against a plan already compressed 120×, which is true real time — and its log's first line, `E1 N=1 reps=8 00:39:40`, names the run directory the E1 artefact's first row still points at. The arithmetic above and the flag agree.

We report that as a check rather than as the answer, because the order matters. The rate was determined from the measurements while the metadata was genuinely unavailable, and the script surfaced afterwards, by luck, from a machine that was about to be switched off. What the agreement establishes is not that E1's provenance was fine all along — it was not — but that the recovery method returns the right answer when there is an answer to check it against. That is the one thing a physical-constraint argument can rarely demonstrate about itself.

The same script also explains a feature we had described and not accounted for. It passes `--max-t-sim 2`: two seconds of match time per run. The median of seven matched events that makes E1 under-powered, and that the main text's attribution section treats as an opening burst, is that window. The burst is real and is why those seven events are the densest in the match; the reason there were only seven is that the campaign asked for two seconds.

We leave the disclosure standing rather than deleting it, for two reasons. The recovery was available only because the artefact we were chasing happened to be a wall-clock constant with a sharp signature, and the confirmation only because a virtual machine had not yet been destroyed; a reader should not have to reconstruct an experimental parameter by arithmetic on its results, and next time there may be nothing to reconstruct it from and nothing left to check it against. And the exercise makes the protocol rule concrete: what was missing was one number, written down once.

What the gap touched. The audit is unaffected either way: a negative transport is causally impossible at any replay rate, and the rejection counts are arithmetic on surviving per-event data. The withdrawal in the main text's attribution section is unaffected by construction — we give the argument in a form that holds at every candidate rate. The rules of the main text's rules section were measured after the pre-flight existed, at a derived and verified rate. the main text's powered-transport section's external validity is restored by the recovery above, and the recovered script now documents it as well — though the inference stood on its own before the script appeared, and is what we would still be relying on had the machine been released a day earlier. the main text's protocol section carries the rule that would have made all of this unnecessary.

2 S2: The load-axis post-mortem, in full

Moved from the main text's mixture development: the load-axis post-mortem in full. The main text keeps the admissions and the numbers.

Both load-axis predictions failed, including ours. The knee sweep placed conditions at achieved ρ of 0.881, 0.920, 0.950, 0.970 and 0.990, where the candidate forms diverge most; the earlier ladder stopped at 0.878 and everything above it collapsed onto a degenerate $\rho = 1.000$, which is why the main text's rules section could not settle the question. With those points in hand, M/G/1 fits *worse than the mean* ($R^2 = -0.05$ against a fitted exponential's 0.93). The withdrawal does not merely stand: the forms are now separated, and M/G/1 is the one ruled out.

The bounded prediction fared better but also failed. Over $\rho = 0.881$ to 0.990 the rate grew by a factor of 1.44. M/G/1 predicted 13.8; the bracket we registered before the sweep predicted 2.45–3.07. The rate saturates, as a bounded mechanism requires — but harder than our own extrapolation allowed, and a prediction that misses low is still a prediction that missed. We report it as a failure rather than as the nearer of two misses.

The two failures have one cause, and E-A5 names it. Our bracket was still parameterised in ρ, through $p = \rho^C$; E-A5 shows ρ does not determine occupancy. What survives is the mechanism — established by manipulation — and not any curve in ρ, ours included.

3 S3: The campaign ledger schema

One row per cell of every instrumented-benchmark campaign, `docs/results/external_campaigns_index.csv`:

- `campaign`, `cell` — sub-campaign directory and cell name; the cell name encodes the swept axis (`l` load percent, `s` message size, `r` producer rate) and replicate number, e.g. `r625_rep3`.
- `level` — the numeric value of the swept axis.
- `valid` — 1 if the cell's counts are usable under the pre-stated rule; 0 rows are retained with a reason, never deleted.
- `count_source` — `shutdown_hook` for exact end-of-run counters; `periodic_quantised` for in-run lines (quantised to 10^4, excluded from analysis); `none_in_log` when no counters were written.
- `kept`, `discarded_zero`, `discarded_negative` — the instrumented guard's three counters, separated by sign.
- `log_bytes`, `log_sha256` — size and hash of the run log the row was built from, so any row can be re-derived and checked against its source.

The Python-harness campaigns (no JVM, Section on generality in the main text) use the same cell naming and publish `docs/results/external/harness_results.csv` with per-cell counts, sign-separated discards, and pacer-jitter percentiles.

4 S4: OpenMessaging distributed mode, failure diagnostics

Eleven attempts in total never produced a measurement. The three diagnosed attempts of the referee campaign (chain 18, block E) fail identically at OMB commit `5b1fa70`: `java.util.concurrent.CompletionException` wrapping `java.lang.IllegalArgumentException` in the coordinator–worker path, before any workload starts. Per attempt, the version, full coordinator and worker logs, and the extracted signature are archived in `docs/results/external/dist_diag/`; a draft upstream report is at `docs/omb_distributed_issue.md` (not filed — filing is the author's action).

Moved from the main text (internal review, round 1). The cross-host case is the one in which OMB's subtraction spans two clocks, and its own distributed mode never completed a run in our hands — eleven attempts, all failing identically inside its coordinator-to-worker protocol, including with no background load at all, which refutes our own load-starvation hypothesis. Its cross-host exposure therefore rests on source audit, the pattern itself measured across two clocks in the main text (per-attempt record and draft upstream report: supplementary material S4). We do not claim

Table 1. The same runs reproduce both published answers. Transport medians (ms) from the powered campaign, computed over every event and over each run's first seven events in emission order — the window E1 saw. The all-events columns reproduce the powered 0.41 ms shift; the prologue columns reproduce E1's near-equality (the main text) in both shift and absolute level. The window, not the brokers, accounts for the disagreement.

		All events			First 7 (prologue)		
N	Runs	Kafka	Redis	Shift	Kafka	Redis	Shift
1	5	0.476	0.095	0.381	0.831	0.743	0.088
9	45	0.533	0.116	0.417	0.939	0.810	0.129
12	55	0.527	0.113	0.414	1.018	0.769	0.248

any published result obtained with this benchmark is wrong: exposure depends on a deployment's path latency and producer rate, and we audited none — why we recommend publishing a retention rate as routine practice, at the cost of one number.

5 S5: The E1 reconciliation, in full

Moved from the main text's powered-transport section: the complete windowed re-analysis showing E1's near-equality is the observation window and nothing else.

Why E1 saw near-equality, tested directly. The two campaigns disagree by more than power: E1 puts both systems near 0.8 ms and within 0.1 ms of each other, while the powered campaign puts them at 0.53 and 0.11 ms and 0.41 ms apart. Attributing that to seven events is an explanation only if it survives being checked, so we checked it. If the difference is the observation window and nothing else, then restricting the *powered* runs to their first seven events — in emission order, the same events E1 would have caught — must reproduce E1, and the same runs must give the powered answer over their full length. That is a prediction with one free parameter fixed in advance, and it can fail.

It does not fail (Table 1). Over all events the runs give shifts of 0.381, 0.417 and 0.414 ms, reproducing the powered 0.41 ms. Over their first seven they give 0.088, 0.129 and 0.248 ms — E1's wandering near-equality, and of the same magnitude as E1's own 0.021, 0.116 and 0.053 ms. The *absolute* medians match as well, which the shift alone does not force: the prologue puts Kafka at 0.83–1.02 ms and Redis at 0.74–0.81 ms, against E1's 0.79–1.00 and 0.72–0.86. One set of runs, one rate, one protocol, two windows, both published answers.

So E1 is not wrong and the campaigns do not contradict each other. E1 measured the opening burst, during which both systems are slow for the reason the main text's attribution section establishes — topic auto-creation, connection setup, and the first blocking `produce()` — and that shared start-up cost swamps a 0.41 ms difference. As the run lengthens the cost dilutes, and the brokers' steady-state separation emerges. The seven-event corpus answered a question about start-up while appearing to answer one about brokers, which is the same failure of window selection as the main text's attribution section, one level up.

6 S6. The broker comparison in full (moved from the main text)

What this claim rests on. We state the evidential basis first, because an earlier version of this section rested it on the wrong corpus. The load-bearing measurement is a powered replication at a replay rate that is *derived from the plan and verified against wall time before the campaign runs* — a documented setting, not an inferred one — over a median of 125 audit-surviving events per run. The original E1 corpus (Table 2) is reported afterwards, as the historical measurement it is, for two reasons that disqualify it from carrying the claim: its replay rate is an inference — recovered in the main text as true real time, but inferred from the runs rather than recorded — and it matched a *median of seven events per run*, the same seven-event opening burst as the scheduling lag we withdraw in the main text. A sub-millisecond difference cannot be resolved on seven events, and E1 does not resolve one: its Hodges–Lehmann shifts wander (0.021, 0.116, 0.053 ms) and its N=1 estimators disagree. That E1 finds the two systems "equal" is as much a statement about seven events as about the brokers. What E1 does establish, and what the powered campaign confirms, is the *concurrency* result: Kruskal–Wallis finds no significant effect of concurrency in either backend ($p = 0.061$ for Kafka, $p = 0.091$ for Redis), and neither system degrades across the range.

The powered measurement. We measured transport directly, at a replay rate derived from the plan and verified against wall time, over a median of 125 events per run rather than seven, at $N \in \{1, 9, 12\}$ with 15 replicates each (Table 3). The numbers are computed over audit-surviving runs only — the campaign's own retention, the gate's effect on every estimate (at most 0.003 ms on the shift) and the worst-case imputation flip point are Table 22 in S34 — and the instrument stamps in microseconds on one clock, so neither failure mode of this paper touches the numbers that follow. The picture is sharper and it is not the one E1 painted. Broker transport is Kafka ≈ 0.54 ms against Redis ≈ 0.11 ms: a Hodges–Lehmann shift of 0.41 ms, Kafka slower, with a bootstrap 90% interval of width ±0.006 ms and $p < 10^{-26}$ at every N. The two systems are *not* statistically indistinguishable; Redis's in-memory `XADD` really is several times faster per operation than Kafka's replicated-log append, which is the direction the grey-literature comparisons report.

And yet the same three estimators — Welch, percentile bootstrap and Hodges–Lehmann — agree that the difference is *equivalent within one millisecond* at every N, because that shift is comfortably inside that margin. Both statements are true and neither is idle: against the seconds-scale human annotation latency that dominates any live sports feed's end-to-end budget, a broker difference is parts in 100,000 of the end-to-end path, so for choosing a broker it is noise; but it is a real, tight, reproducible effect that a properly powered instrument resolves and a seven-event one cannot. The shift is also flat across concurrency (0.408, 0.416, 0.417 ms at $N = 1, 9, 12$, audit-gated; S34), so neither system degrades. A part of even this residual is the instrument rather than the broker: the main text shows that 0.07 ms of the gap is the asymmetric acknowledgement stamp, leaving a true broker-transport difference near 0.34 ms. That 0.07 ms is a concrete instance of the blind spot of the main text: it is a bias smaller than the quantity measured, so it never turns a value negative and the consistency check passed both corpora that carried it. The check guarantees a corpus is not *catastrophically* wrong; resolving a 0.07 ms instrument artefact still took a controlled experiment.

An independent replication confirms it. A second powered campaign, run days apart with the same protocol, gives Hodges–Lehmann shifts of 0.397, 0.412 and 0.413 ms at $N = 1, 9, 12$ — inside the 90% interval of the first campaign at every level, with overlapping intervals throughout. The 0.41 ms advantage is not a fluctuation of one campaign.

This refines E1 rather than contradicting it, and it sharpens the contradiction with the first-answer table in the main text: the accelerated corpus reported Redis *degrading* with concurrency; the powered real-time measurement finds Redis uniformly *faster* and flat. The reversal is the audit's doing, and the measurement's.

Why E1 saw near-equality, tested directly. If the disagreement between E1 (< 0.1 ms apart) and the powered campaign (0.41 ms apart) is the seven-event window and nothing else, then restricting the powered runs to their first seven events must reproduce E1, and the full runs must reproduce the powered answer. Both hold: full-length shifts 0.381, 0.417, 0.414 ms against the powered 0.41; first-seven shifts 0.088–0.248 ms against E1's 0.021–0.116, with absolute medians matching as well. The window explains the disagreement with one parameter fixed in advance; the full test, including the start-up decomposition it rests on, is supplementary material S5.

Table 2. **Historical corpus; the scheduling-lag columns are withdrawn.** End-to-end delay and its decomposition, Testbed B, after the consistency check (164 of 201 runs retained). Medians, ms. **The TTI and scheduling-lag columns record a per-run start-up cost, not a per-event property, and are retained only as evidence for the withdrawal in the main text; they should not be read as system behaviour.** The transport columns are computed over the same seven-event opening burst, and so measure start-up rather than steady state: supplementary material S5 reproduces them from the powered runs by restricting those runs to the same window. For steady-state transport see Table 3. The replay rate was not recorded and is recovered as true real time in the main text.

	TTI		Sched. lag		Transport	
N	Kafka	Redis	Kafka	Redis	Kafka	Redis
1	105.3	5.0	103.0	1.4	0.792	0.721
9	105.5	5.4	103.0	1.9	0.802	0.807
10	105.3	5.8	102.8	2.0	1.000	0.855
12	105.7	5.3	102.9	1.7	0.839	0.844

Table 3. Powered transport replication at a verified true-real-time rate, over a median of 125 events per run (not the seven of the main text), audit-gated (S34). Medians in ms; the Hodges–Lehmann shift is $Kafka - Redis$ with a percentile-bootstrap 90% interval. The brokers are equivalent within 1 ms at every N (all three estimators) yet cleanly distinguishable: Redis is ≈ 0.41 ms faster, and the shift does not grow with concurrency.

N	Events/run	Kafka	Redis	HL shift [90% CI]
1	148	0.512	0.102	0.408 [0.389, 0.419]
9	112	0.542	0.120	0.416 [0.408, 0.423]
12	121	0.540	0.114	0.417 [0.408, 0.424]

7 S7. The end-to-end withdrawal in full (moved from the main text)

TTI and transport give opposite answers in Table 2. Of Kafka's 105.5 ms, 102.9 ms is producer scheduling lag against Redis's 1.8 ms, while broker transport is 0.84 and 0.80 ms. An earlier version of this paper reported that twentyfold end-to-end gap as a finding, attributed it to the client's sender thread waking from idle, and built a practitioner recommendation on it.

That gap does not survive re-measurement, and the reason is a second instance of this paper's own thesis. We report it at length for the same reason we reported the first, and we retain the figure that once carried it (Figure 2) rather than deleting the evidence.

The offset is a start-up cost, not a per-event constant. Re-measuring at N=1 on the same driver and the same broker, at a replay rate derived from the plan and verified against elapsed wall time, gives a median producer scheduling lag of 1.59 ms for Kafka, not 103 ms — with a *maximum* of 103.5 ms. Two independent instrumentation paths agree: the per-event loop trace and the per-run summary. The 103 ms is real, but it is paid once per run, not once per event.

We cannot call this a repetition of E1's condition, because E1's replay rate is not recoverable (the main text). That is why the argument below does not rest on the comparison between the two campaigns at all. It rests on a sweep that is internally controlled — one campaign, one rate, one match, only the window varying — and on an arithmetic relation that holds at every rate E1 could have used.

A controlled test that separates the two. A median cannot distinguish a per-run cost from a per-event one, because how much of the cost the median sees depends on how many events the run contains — which is precisely what misled us. The quantity that *does* separate them is the *number* of affected events per run. A per-run cost predicts that number is fixed however long the run gets; a per-event constant predicts it grows with the run. We therefore replayed the same match at true real time under three observation windows, holding everything else fixed, and counted events waking more than 50 ms late (Table 4).

The count does not move (Figure 1). Going from the 60 s window to the 600 s window multiplies the events emitted by 8.9 — 57 to 507 — while the number waking more than 50 ms late stays at exactly four and exactly one send blocks, so the share of events paying the cost falls from 7.0% to 0.8%. The median is flat at 1.6 ms throughout and the maximum flat at 103.5 ms: the cost is present in every run and dilutes in every run, which is the signature of something paid once. Had it been a per-event constant, the count would have grown with the window and the median would have stayed at 103 ms.

A once-per-producer cost of this kind is documented upstream: a Kafka producer performs a metadata lookup before its first send to a topic it has not yet cached [1]. We did not isolate that call, so we do not claim it *is* the mechanism — the window sweep constrains the cost's *shape*, not its origin. We note it because the shape we measure and the shape the client documents agree, and because a reader deciding whether this generalises beyond our testbed should know the behaviour is a property of the client rather than of our rig.

Redis was run in the same sweep on the same events, with the same loop trace, and has *no* events waking more than 50 ms late and *no* blocking send at any window. Its worst single event anywhere is under 25 ms. Instrumenting both arms identically matters here for a reason internal to this paper: an earlier version of this table reported Redis's counts as zero when its producer in fact carried no trace, which is the absence of a measurement dressed as one — exactly the failure the paper is about. We added the trace to the Redis producer and re-ran both arms together.

The mechanism, read directly off the loop trace. The per-event trace shows the same five-event signature in every Kafka run, at every window:

event	wake late (ms)	send blocked (ms)
0	0.17	**102.60**
1	**102.96**	0.07
2	**103.08**	0.05
3	**103.17**	0.04
4	**103.30**	0.05
5	0.94	0.27
6	1.42	0.06

The first `produce()` blocks for 102.6 ms fetching metadata and creating the topic. The replay loop is single-threaded, so every event due while it is blocked wakes roughly 103 ms late and then sends in tens of microseconds. There are exactly four of them, and the reason is the sport rather than the harness: this plan's first five events all carry t_{sim}=0, because a kickoff is recorded as a pass, a ball receipt and a carry inside the same second. That is not particular to this match — every one of the eleven plans in our corpus opens with at least five events at t_{sim}=0. A football feed therefore delivers its densest burst precisely when the producer is coldest, which is the worst possible alignment for a start-up cost and the reason it lands in the median rather than the tail. From event 5 the loop is in steady state at about 1 ms. One cost, paid once, is charged to five events; the sixth onward pays nothing. Redis shows no such prologue because `XADD` to a new stream creates it in the same round trip.

That closes it. Five events carry the cost, seven were matched, and the median of seven values of which at least four are 103 ms *is* 103 ms. The published figure is the arithmetic of its own prologue.

Why the original corpus could not see that, and how we know. The E1 runs behind Table 2 replayed the first 600 s of match time, which in this plan holds **507 events**. They matched a **median of seven** — 745 events across 100 Kafka runs, a match rate near one percent. The window was not the problem; the join was. And the seven that survived it are not a random seven.

That last claim is arithmetic rather than inference. Those runs report a median producer scheduling lag of 102.93 ms over seven values, and a median of seven values at 102.93 ms requires *at least four of the seven* to be 102.93 ms or greater. The loop trace says exactly how many events per run are ever that late: four (Table 4), and always the same four — those due while the first send blocks. The matched set must therefore consist almost entirely of the prologue. The plan's first five events are all stamped at t_{sim}=0, so they are the first candidates a consumer sees and the first a lossy join keeps.

The failure thus compounds: a sample of seven is small, but a sample of seven *selected towards the contaminated events* is worse than small. Redis reports 1.75 ms on the same events because opening a stream with `XADD` does not pay what Kafka's first send pays in metadata fetch and topic creation, so the same selection costs it nothing.

Why not knowing E1's replay rate does not matter here. The blocking first send lasts 102.6 ms of *wall* time, so at a replay rate of R times real time it spans $0.103R$ seconds of *match* time. Evaluating that against the plan for each rate E1 could have run at:

Replay rate	Match time spanned	Events inside the prologue
1×	0.10 s	5
120×	12.3 s	16
1200×	123.1 s	112

E1 matched a median of seven events per run. At every one of the three candidate rates the prologue already contains at least four events — enough to carry the median of seven — so the matched sample's median is the start-up cost, which is what the corpus reports. The conclusion is therefore insensitive to the replay rate; the main text goes on to recover that rate from the same start-up cost, but the withdrawal does not depend on the recovery.

This also explains the three properties we previously offered as evidence, and each of them is equally consistent with a start-up cost: a per-run constant looks *constant* within a run; it is *concurrency-invariant* because every run pays exactly one regardless of N; and it is *rate-dependent* because accelerated replay packs more events into the window and dilutes it. We read three signatures of a per-event mechanism into measurements that a per-run mechanism explains at least as well.

The general lesson. The integrity check of the main text does not catch this. Every one of those runs is causally consistent; nothing is negative; the medians are stable to three significant figures across a hundred runs. What went wrong is that a median over seven events was reported as a property of the system rather than of the window, and the tight agreement across runs made it look robust precisely because the artefact is deterministic. A benchmark can be internally consistent, gate-clean, and still measure its own start-up. We therefore add to the main text the requirement to report events-per-run alongside any latency percentile, since a percentile over single-digit samples is a statement about the harness.

What we claim instead. On broker transport — the quantity that survived the main text and is measured over the same events for both systems — Kafka and Redis remain equivalent within one millisecond (the main text). We make no end-to-end claim. The twentyfold figure is withdrawn.

Table 4. The observation-window sweep that separates a per-run cost from a per-event one. Same match, same driver, same broker, N=1, true real-time replay; only the window changes. Both producers are loop-traced, so the counts for the two systems come from the same instrument. Events emitted grow 8.9× across the sweep, but the number of Kafka events waking more than 50 ms late does not move, and exactly one send blocks in every run; Redis has neither at any window. Counts are medians over three replicates from the per-event trace; percentiles come from the per-run summary.

	Window	Events		Scheduling lag (ms)		Events > 50	Blocking
	(s)	emitted	matched	median	max	ms late	sends
Kafka	60	57	57	1.56	103.4	4	1
	180	148	148	1.61	103.5	4	1
	600	507	465	1.58	103.5	4	1
Redis	60	57	57	1.50	17.4	0	0
	180	148	148	1.56	20.4	0	0
	600	507	465	1.53	24.8	0	0

The benchmark's own output corroborates the insensitivity (moved from the main text, internal review, round 1). The benchmark's own output corroborates this without our instrumentation: across eight runs all 32 reported percentiles are whole milliseconds (only the averages, means of integers, are fractional), and three runs report p50 = p95 = p99 = max = 1.0 — not a narrow distribution but one with a single value in it, printed to three decimal places.

8 S8. The first corpus's prior corrections (moved from the main text)

We present the first result set in full rather than describing it, because a reader cannot otherwise judge how convincing an invalid corpus looks.

8.1 The result

On Testbed A, replaying distinct real matches at ten times real speed across one, five and ten feeds with 15–30 runs per cell and both producers pipelined, broker transport separated the two systems cleanly (the main text). Every comparison was significant after correction and at $N \geq 5$ the run distributions did not overlap.

8.2 The checks it had already passed

The corpus was not naive. It was the product of five earlier corrections, each of which had reversed the apparent winner and each found by investigating a result that looked wrong:

(1) *A process-relative clock.* Cross-process timestamps taken with `perf_counter_ns`, whose origin is per-process, injected each run's consumer start-up offset into every measurement. Fixed with a shared wall-clock epoch.
(2) *A saturating replay speed.* At 120× the producer could not keep pace, inflating scheduling lag to tens of seconds.
(3) *An asymmetric load generator.* The Kafka producer blocked on a broker round trip per event while the Redis producer dispatched asynchronously. Median TTI at one feed fell from 1,644 ms to 16 ms once the Kafka producer was pipelined.
(4) *A concurrency axis that was covertly a throughput axis*, from the merged-plan design described in the main text.
(5) *A heavy tail contaminating every mean.* Kafka's end-to-end 95th percentile sat at 99–105 ms at every concurrency level while its median moved from 1.8 to 7.2 ms. A 95th percentile that ignores load is not system behaviour; decomposition located it in scheduling lag, not transport.

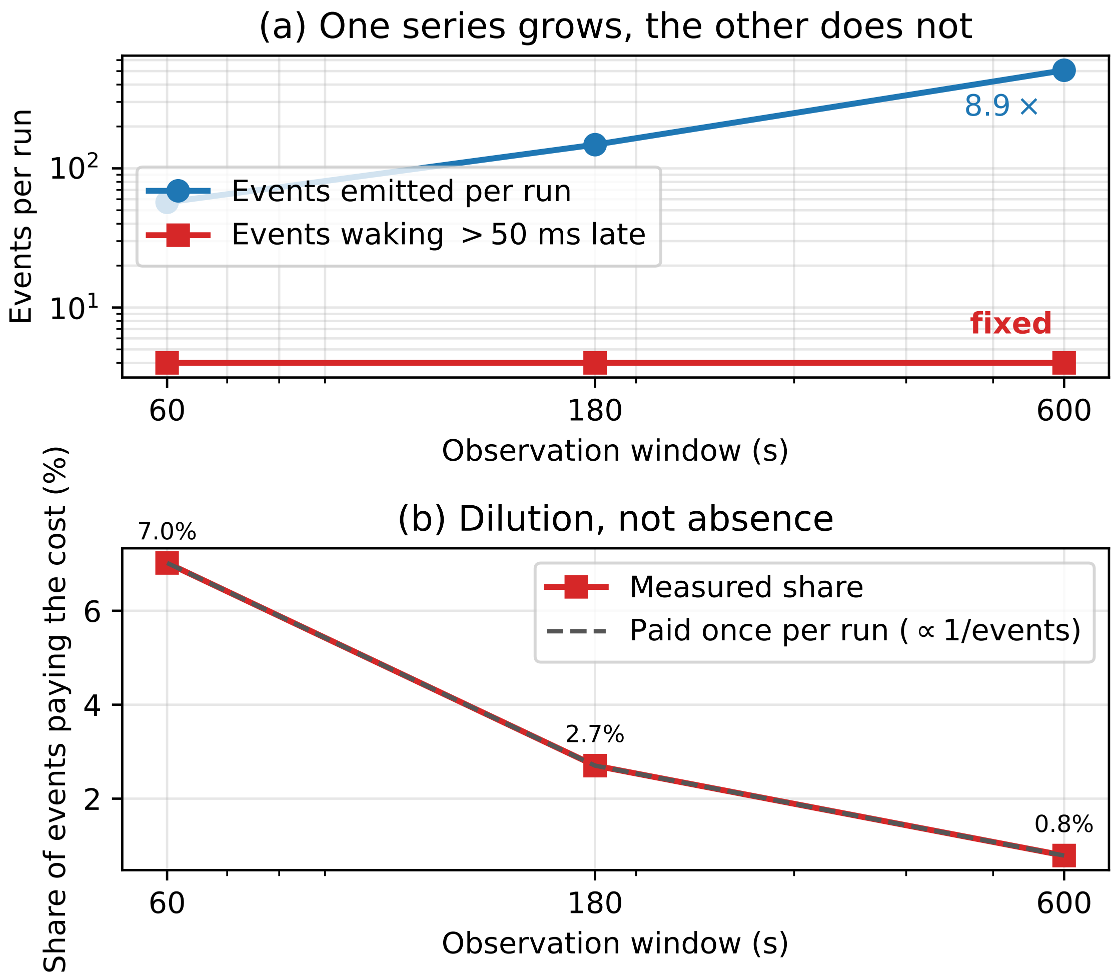

Fig. 1. The window sweep, as a picture. (a) Events emitted per run grow 8.9× while the number waking more than 50 ms late stays at four — a per-event constant would put the two series on parallel lines. (b) The same fact as a proportion: the share of events paying the cost falls as 1/events, the dilution a once-per-run cost predicts. Redis, on the same events, has no affected events at any window.

We also verified the comparison was not biased by message size or protocol: payloads are near-identical (median ≈170 bytes) and serialisation costs are negligible and comparable.

A sixth problem shaped our later protocol. Our first run of the acknowledgement experiment of the main text reported no improvement whatsoever — a clean refutation of our own hypothesis. The treatment had never reached the consumer: the orchestration script accepted the option but, through a failed edit, never passed it on. We found this only by deliberately supplying an invalid option and observing that the consumer did not reject it. A null produced by an unapplied treatment is indistinguishable in the data from a genuine one, so every intervention reported here carries a manipulation check that aborts the campaign if the treatment is not accepted.

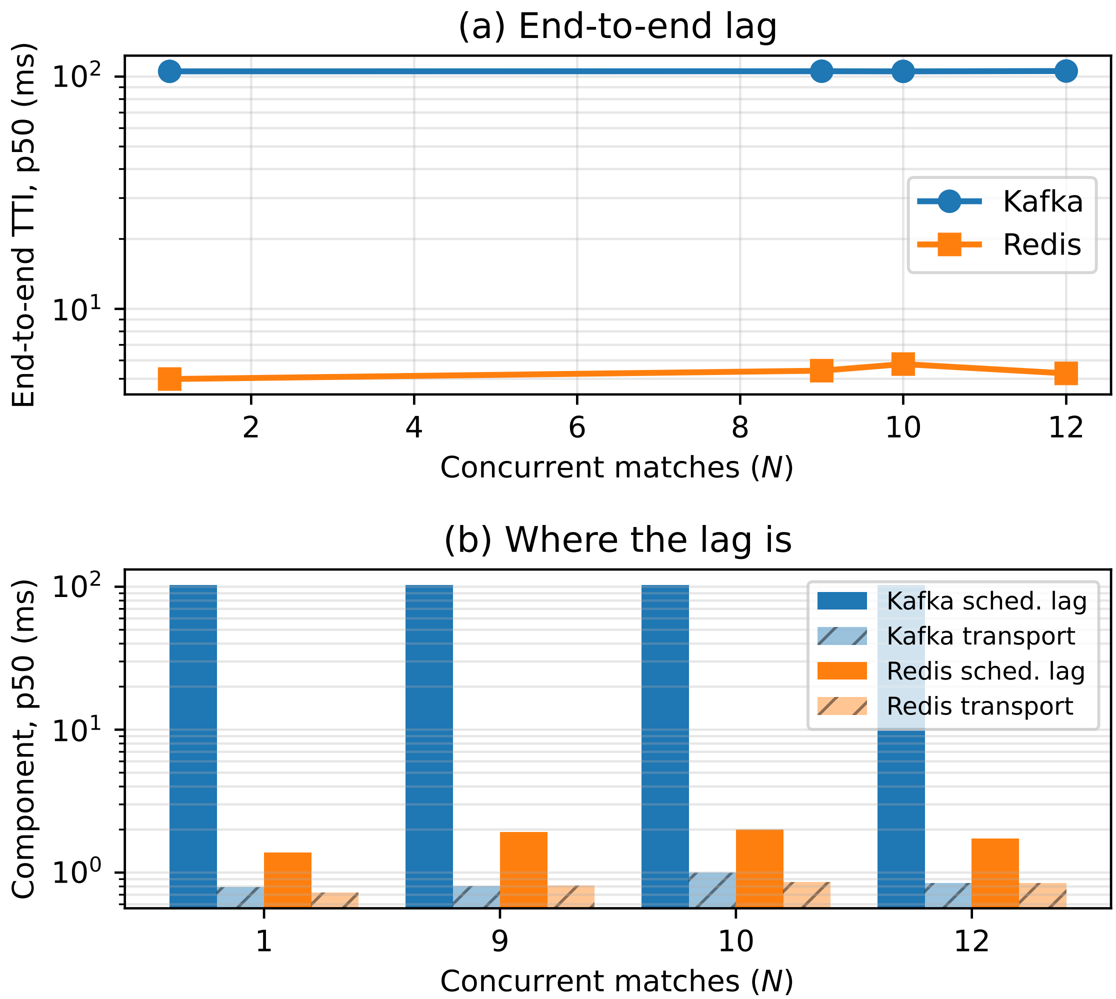

Fig. 2. **Withdrawn.** (a) End-to-end delay against concurrency as originally measured. (b) The decomposition that appeared to place the whole gap in producer scheduling lag. Both panels are retained as evidence for the main text: the runs behind them matched a median of seven events each, so the apparent per-event offset is a per-run start-up cost. Broker transport, the lower series in (b), is unaffected and is what the paper claims.

The point of the list is not the corrections. It is that a corpus which had survived six of them, a fairness audit, and a full battery of rank tests, effect sizes and multiplicity correction, was still entirely invalid, and nothing in its output said so.

9 S9. Open questions from the comprehensive campaign (moved from the main text)

Three observations from the campaign are consistent with a single conjecture — a per-send jitter kernel whose width in cell units grows with the numerator p of Δ/τ — and none of them is established. We separate them from the results above deliberately: they are post-hoc, the pacer they implicate is measured only in the non-JVM replication of the main text, and one of our own follow-up tests cuts against the conjecture.

Displacement toward θ. Replicates sit off their grid vertices toward the continuous value, from under a point to complete detachment. The two arms that detached outright (a majority of replicates more than 3 points from every vertex) are 300 msg/s in the evening pass and 700 msg/s — both with $p = 10$; no arm with $p \leq 8$ detached, and the same evening that 300 msg/s detached, 625 msg/s ($p = 8$) produced its cleanest two-branch split of the study. Suggestive; a census of two is not a law.

Mobility between passes, now measured under control. The same 300 msg/s configuration was branch-adjacent in the morning and detached in the evening; passes were hours apart, so epoch and arm were confounded. The controlled test interleaves the two arms A/B/A/B within one session — four pairs of (300, 625) msg/s — and **both arms hold their grids**: 300 msg/s at 37.40, 38.21, 39.86 on the lower vertex and 67.41 on the upper; 625 msg/s at 42.97 and 55.91–60.18 (the `ultimate_alt` cells of the artefact). No arm detached. The detachment is therefore a property of the *epoch*, not of the arm, and the numerator census above loses its same-evening contrast — which cuts the kernel conjecture's strongest support and is reported accordingly. What remains established is that some sessions produce a detached regime and others do not, at fixed configuration; the disciplined clock is exonerated throughout (residual 0.013 ppm), and the pacing loop's state remains the open suspect.

Branch suppression. The formal residual analysis (the main text) finds mean retention *below* θ at every $q \geq 3$ arm, and the rank correlation of |residual| with p is negative (−0.8), not the positive association a displacement-scaling kernel predicts — the sign is dominated by branch-weight sampling at $q = 1$. What the residuals do support is minority-branch suppression: a kernel narrower than the half-cell keeps runs on the nearest vertex, depressing the upper-branch weight below $\mathrm{frac}(q\theta)$, which would also account for the 1250 msg/s miss above. A quantitative displacement prediction from a measured jitter statistic does not yet exist; the harness of the main text records the statistic per run and is where one would be built.

10 S10. The distributed-mode attempts in full (moved from the main text)

The cross-host case is the one in which OMB's subtraction spans two clocks, and therefore the one in which its guard could discard a genuine causality violation. **OMB's own distributed mode cannot report a measurement of it.** Across eleven attempts — eight at three background-load levels including none at all, which refutes our own hypothesis that CPU starvation was the cause, and three further attempts run with full diagnostics for this revision — the benchmark's distributed mode never completed a run, failing each time inside its own coordinator-to-worker protocol rather than in our instrumentation or configuration. The validity gate wrote `valid=0` with a reason on every attempt; per-attempt versions, logs and failure signatures are archived in the artefact and summarised in supplementary material S4, and a draft upstream report accompanies them.

The exposure of OMB-in-distributed-mode is therefore established by source audit — the timestamp is written on the producer's host and read on the consumer's, and the guard drops whatever the difference produces — and *not* by observing that binary. What *is* now observed is the pattern itself across two hosts: the harness of the main text runs the identical stamp-subtract-guard logic over two disciplined clocks and measures zero negatives, an intact grid, and a deleted fraction coupled to clock-offset drift. The two statements are complementary and we keep them distinct: the pattern's cross-host behaviour is measured; OMB's own distributed binary remains unmeasurable by us.

What we do not claim. That any published result obtained with this benchmark is wrong. We have shown that the deletion happens, that it is large, that it is irreproducible, and that it is unreported. Whether it has affected a specific published measurement depends on that measurement's path latency and producer rate, and we have audited none. That asymmetry is the reason we recommend publishing a retention rate as routine practice: its cost is one number, and the alternative is a benchmark that cannot report its own failure.

11 S11. The workload in full (moved from the main text)

The workload is a live football event feed, and we describe it only as far as the results require. Full characterisation of 3,315 matches from the StatsBomb open dataset [2], spanning 52 competition-seasons from 2003 to 2023, is available in the artefact; event data of this kind is collected by trained human operators, whose accuracy has been studied [3, 4].

Three properties of the workload drive every experimental choice (Table 5). We name, for each, the later result that depends on it, because a setting section that cannot say what it is for should not be in the paper. A reader who wants only the measurement contribution can note that the workload supplies three things a synthetic generator would not have supplied — a sparse arrival process, a burst at a known instant, and an empirically bounded concurrency — and skip to the main text.

It is sparse. A match produces a median of 3,507 events over roughly 8,412 seconds, a mean rate of **0.415** events per second. This is four orders of magnitude below the rate at which either broker begins to strain, and it is the property that makes the main text's result possible.

It is bursty, and one burst has a known instant. Over a sliding ten-second window the same feeds peak at 2.70 events per second, a peak-to-mean ratio of 6.45; half of all inter-arrival gaps are one second or shorter. The feed is idle most of the time and busy in short bursts. One of those bursts is special: a kickoff is recorded as a pass, a ball receipt and a carry inside the same second, so *every* match in our corpus opens with at least five events sharing t_{sim}=0. That coincidence — the densest burst arriving when the producer is coldest — is what turns a one-off client start-up cost into the median of a short run, and it is the mechanism behind the withdrawal in the main text. A synthetic Poisson generator has no kickoff, and would not have produced it.

Its concurrency is bounded and knowable. Match records carry kick-off times, so the number of simultaneous feeds is empirical rather than swept. Across 2,346 kick-off instants the largest carries **12** simultaneous matches, and at most **21** are in play at once. Domestic leagues schedule the final matchday simultaneously by rule, so a 20-team league plays ten concurrent matches. We therefore benchmark at $N \in \{1, 9, 10, 12\}$: the median slot, the upper quartile, the structural league bound, and the observed maximum. This is what keeps the concurrency axis of the main text from being an arbitrary sweep — the levels are measured facts about the domain, not round numbers. Two limits apply. The dataset is a sample, so observed simultaneity is a lower bound; and the bound is per league, so a platform serving many competitions faces their sum.

Taken together these also explain why the broker question turned out to be the least interesting thing we could have asked, and we would rather say so here than let a reader discover it in the main text. At 0.415 events per second and at most a dozen concurrent feeds, this workload sits four orders of magnitude below where either broker strains. The domain that made the failure visible is the same domain in which the original question has a boring answer.

12 S13. The commensurability account's corrections in full (moved from the main text)

We report the correction because the first version was weaker and we published it to ourselves. Treating $100/q$ as a point prediction made the even denominators look like failures, and we introduced an exclusion for them — an arm

Table 5. The workload, from 3,315 matches. Medians across matches; peak rate is the maximum over any sliding ten-second window.

Property	Value
Events per match	3,507
Match duration	8,412 s
Mean arrival rate	0.415 ev/s
Peak rate (10 s window)	2.70 ev/s
Peak-to-mean ratio	6.45
Median inter-arrival gap	1.0 s
Distinct kick-off instants	2,346
Max simultaneous kick-offs	12
Max matches in play at once	21

cannot test a grid the continuous value sits on, so $q = 2$ and $q = 4$ were set aside. That exclusion was defensible and pre-registered: it was derived from the $q = 4$ result and committed at 14:09Z on 2026-07-27, and the first cell of the first odd-q arm was measured at 15:06Z, so the odd denominators were run *because* the rule said they would separate. It was also unnecessary. Under the main text an on-grid arm is a prediction of a flat arm, so $q = 2$ and $q = 4$ are evidence *for* the account rather than cases to be excused. We had reached for a law that excluded its exceptions when one that explains them was available.

What exposed it was checking whether the exclusion survived re-estimating one of its own inputs. The continuous value is not constant across rate — it runs 51.2% at 333 msg/s down to 47.5% at 889, a trend of 4.15 points against a replicate noise of 1.55 — and under a rate-local estimate $q = 4$ ceased to be degenerate and became a failure. A classification that flips on that choice is not a classification. Measuring position against the cell half-width instead of a fixed noise floor removes the sensitivity: every call in the main text is the same under either estimate.

A second prediction of ours failed, and no reading of it rescues the number. We recorded, before running it, that $q = 5$ would put 2 of 5 phases across a boundary and so retain about 40%. The five replicates gave a median of 46.6% — inside the incommensurate range and 6.6 points from what we wrote down. The error was again naming one branch of a two-branch prediction: with T_{true}/τ at a half, 2 *or* 3 phases cross depending on where a run's initial phase falls, so a replicate is predicted at 40% or 60% and the average of many is near 50. The five values — 40.7, 42.1, 46.6, 55.5, 58.6 — are bimodal about exactly those two branches, which is what the account requires and what no continuous account produces. The grid held; the point we picked on it did not.

Where the account is approximate, stated rather than trimmed. Three observations sit outside it. The $q = 1$ arm at 250 msg/s spreads 58.6 points where the cell width is 100, and one replicate at 500 msg/s sits at 18.0; a run confined to a single phase can produce neither. We read this as the limit of the idealisation: a nominal rate is $q = 1$ only if the pacing is exact for the whole run, and over three minutes a drift of a few parts per million walks the phase across a boundary part-way through. The grid that governs retention is the one the pacing realises, not the one the nominal rate implies. Third, $q = 2$'s spread of 17.6 rests on a single replicate at 68.0; the other four span 3.1 points, which is the incommensurate noise level and exactly what an on-grid arm should show. We report the spread over all five rather than the four that agree.

13 S12. The two-state model commentary and the tracing instrument checks (moved from the main text)

What it accounts for. Clustering follows by construction: being preempted is an *interval*, so the events inside it are corrupted together and a two-state process cannot help but cluster. The 60× growth in tail weight against 5× in core width is two different parameters moving — σ_c belongs to the running state, p to the scheduler. The scale family fails because a scale family has one parameter and here two move at very different rates.

A prediction that points the opposite way. The two structures disagree geometrically. A scale family changes σ, so its tail curves *rescale horizontally*; the main text changes p, so its curves *shift vertically and stay parallel* on a log scale. Taking the log ratio of two conditions' tail masses at several thresholds, the two-state model predicts a constant, independent of the threshold.

Restricting to tail estimates backed by at least 20 events — nine far-tail points resting on five to eight events are excluded, as they carry log-scale errors larger than the effect — the median spread of that log ratio across conditions is **0.29**, a factor of 1.34, with a worst case of 0.91. Set against the 23× failure of the scale-family collapse, the tail curves are close to parallel. The two conditions with the largest spreads sit at intermediate load, where the residual distribution is plausibly still shifting, so the honest statement is $\Pr[\mathrm{inv}] = p(\rho)\, S(c;\rho)$ with p varying fast and S slowly: separability is a good first-order description rather than an exact law.

What this is worth, and what it is not. It is mechanistic — occupancy names the moving part — and actionable: if the failure is occupancy, the mitigation is a stamping thread the workload cannot preempt, not a better clock. But we found this by looking at the data; the separability test is exploratory (the main text), so we pre-registered a confirmatory version and ran it: the campaigns below, the decisive one being real-time priority at fixed utilisation. Two states is the smallest model consistent with the data, not a kernel claim.

On the load axis the model is a tautology, and we say so before using it. Read as a function of *ho* alone the model has no content: with p unconstrained, any monotone rate curve can be written $p(\rho)\, S$; the model's content is on the threshold axis, where it passed. Constraining p to a power law fits poorly ($R^2 = 0.65$ in log space). The form that restores the divided-out σ, $\Pr[\mathrm{inv}] = p(\rho)\, S(T_{\mathrm{true}}/\sigma(\rho))$, reaches $R^2 = 0.9905$ against 0.9811 (exponential) and 0.8863 ($(\rho/(1-\rho))^k$) at equal parameters — *and we do not report that lead as a result*: freezing σ at its mean improves the fit to $R^2 = 0.9982$, because σ and ρ move together on this ladder and no fit can credit one over the other. That is a design limit no further analysis of these data can lift.

The instrument check, and what it can and cannot rule out. Tracing a system to measure its scheduling behaviour is exactly the kind of intervention that can create what it observes, so the traced cell has an untraced twin. Our first attempt at this campaign ran with a probe that failed to attach: the histogram was empty, but the inversion rates are valid, and we kept that run as the control. It measured 0.2717 against the traced run's 0.2315 — the traced rate is 14.8% *lower*, two hours apart, same design.

We report that rather than a flattering comparison, because an earlier version of this section compared the traced cell against a different campaign's 88% arm (0.2214) and called the 4.6% gap a clean bill of health. The honest reading is weaker. The ordinary 88% arm has been measured five times across these campaigns and spans 0.221 to 0.305, a factor of 1.38; the traced value sits inside that spread, and so does its untraced twin. So we can say that tracing did not move the rate by more than the drift already present between campaigns, and we cannot say anything tighter than that. A tracing effect smaller than about 15% is not resolvable with this control.

What that limits is the *level*, which no claim here rests on. The comparison in the main text is between two quantities measured in the *same* run — a kernel histogram and a count of negative transports over the same events — so a common perturbation of that run moves both and leaves their agreement intact.

The gate fired on our own replication, and we report the arm it withheld. The check is a rule fixed in advance: a traced arm whose inversion rate differs from its untraced comparator by more than 25% is not compared. The replication's 88% arm measures 0.1956 against the same untraced control's 0.2717 — a drift of 28% — so its verdict is withheld by the same mechanism that admitted the original. Its numbers are in the main text because withholding a comparison is not a reason to hide a measurement, and because the ratio it would have contributed, 1.06, is the closest agreement in the table. Reporting only the arms that passed would leave a reader with three ratios and no idea that one of them is inadmissible under our own rule.

The real-time arm's zero is an artefact of the instrument, and the replication settles it. Every traced real-time arm we have run — three of them, across two campaigns and two load levels — recorded *zero* inversions in 2,985 events. The untraced twin of one of those arms recorded 15. Under that rate the chance of a single traced arm showing none is $3{\times}10^{-7}$, and of all three about 10^{-20}.

We first reported this as unresolved: either the arm genuinely differed or tracing perturbed it, and one measurement could not separate the two. Three can. Attaching BPF to `sched_switch` suppresses the inversions in the real-time arm, and the effect is not subtle — it removes all of them. The direction is consistent with the ordinary arm, where the two traced measurements are the lower two of the six we have at 88%, though there the drift between campaigns is wide enough that the same conclusion cannot be drawn.

This is the paper's own thesis arriving uninvited. We attached an instrument to measure why an instrument was failing, and the second instrument changed the thing it came to observe — in the arm where the quantity was smallest, which is exactly where the main text says a measurement is most exposed. It costs us nothing here, because no claim rests on the real-time arm's level, and it is worth stating plainly rather than leaving in a footnote: a kernel tracer is not a free observer.

14 S15. Configuration in full (moved from the main text)

Table 6 states every setting that can move a latency number, with the reason it holds its value.

Mohammad [5] finds that published Kafka comparisons are frequently not comparable because client configuration and acknowledgement semantics differ silently between the systems being compared. We therefore state every setting that can move a latency number, with the reason it holds the value it does (Table 6). Two of these were not chosen but *learned*: the pipelining setting and the acknowledgement-batching setting each cost us a result before we understood them, and both are the subject of the main text.

The general principle is that the two clients must be equalised on *semantics*, not on parameter names. Kafka and Redis expose different knobs, and matching them syntactically is what produces the asymmetries this paper is about. Concretely, Kafka's producer buffers and pipelines by default while our Redis producer dispatches to a worker pool; leaving `max-inflight` at 1 makes the Kafka client wait for a broker round trip per event while the Redis client does not, which is not a broker difference but a harness difference.

15 S16. Instrumentation of the external benchmark in full (moved from the main text)

The second failure mode is measured in the OpenMessaging Benchmark rather than in our harness, so the instrument is a patch to somebody else's code and its design needs stating.

Table 6. Every configuration that can move a latency number, and why it holds that value. Settings marked *learned* were discovered by a defect they caused, not chosen in advance.

Setting	Value	Why
Kafka producer		
`acks`	`all`	Durability semantics matched to Redis's synchronous `XADD`; weaker acknowledgement would compare different guarantees.
`linger.ms`	0	No artificial batching delay; the workload is sparse, so any linger would dominate the quantity measured.
`max-inflight`	64	*Learned.* At the default 1 the client blocks on a round trip per event, making the comparison one of harnesses rather than brokers (the main text).
`compression`	none	Football events are small JSON; compression would add CPU to the path being timed for no representative benefit.
`ack-stamp`	`callback`	Default, and deliberately asymmetric with Redis; the main text measures what the asymmetry costs by also running `inline`.
Redis producer		
send workers	pool	Non-blocking dispatch so the emission loop keeps schedule; send and acknowledgement are stamped in the worker, preserving a valid transport span.
Consumers		
Redis `ack-batch`	200	*Learned.* Acknowledging per message adds one network round trip per message, which at 20 ms of delay is the difference between 103 ms and 4,138 ms (the main text).
Redis `count`	200	Read batch; matched to the acknowledgement batch so neither dominates.
Kafka `poll-timeout`	1000 ms	Client already amortises fetches; no per-record round trip exists to remove.
Replay		
speed-up	derived	*Learned.* Plans carry a baked-in 120× compression, so the flag is not the rate; it is derived from the plan and verified against wall time before every campaign (the main text).
window	60–600 s	Swept deliberately: the number of events a run contains is itself a variable, and the main text is the reason.

What the patch does, and what it deliberately does not. At the named commit, a sample is admitted to the histogram only by `if (endToEndLatencyMicros > 0)`. We add counters in the existing `else` branch and change nothing else: not the latency computation, not the guard's condition, not any reported statistic. The benchmark's own output is therefore exactly what it would have been unpatched, which is the property that lets us set its published summary beside the retention behind it. The diff is in the artefact.

The counters must distinguish sign, and an earlier version did not. Both a causality violation and a sub-millisecond delivery fail a `> 0` test. A single unsigned total cannot tell them apart, and we initially reported one, drawing a conclusion the sign-separated data later refuted (the main text). The counters are now four — zero, negative, most-negative, kept — and the distinction between them carries the finding.

Counts must be exact, which required two corrections. The counters are declared `static`: an embedded run constructs several `WorkerStats` instances, and per-instance counters silently split the totals between them. Totals are printed once by a JVM shutdown hook rather than read from periodic progress lines, because those fire every 10,000 discards and reading the last one quantises the answer to a multiple of 10,000 — three runs with genuinely different totals all reported 50,000 that way. Every cell records which source its counts came from, and cells whose counts are quantised are excluded from every analysis by rule rather than by inspection.

A validity gate that refuses to report a number. A benchmark that fails to run discards nothing, and a zero from such a run looks exactly like a measured zero. Our first distributed attempt wrote `discarded=0` after the worker died on a classpath fault, and that vacuous zero reached a draft of this paper as a genuine negative result. The campaign now validates before writing: the run must have produced publish-rate output and no failure lines, and if it did not, the row is written `valid=0` with the reason and no count. The gate has since refused eight distributed attempts (the main text), and its refusals are in the artefact alongside its acceptances.

Denominators. OMB warms up for one minute by default, resets its histograms at that boundary, and reports percentiles over the test phase alone; our counters are never reset and would otherwise span both. We make the setting explicit and report a matched sweep with warmup disabled. It changes the sample count per cell from ≈ 120,000 to ≈ 90,000 and leaves the retention fraction unchanged, so the figures computed with warmup included stand.

Campaign design. Retention at a fixed configuration proved not to have a stable central value (the main text), which dictates the design: every claim is made over whole distributions of replicates rather than over a summary of them, and the axes are swept one at a time with everything else held fixed — background load, message size, producer rate, and the warmup setting — because the variable that turned out to matter was one we did not initially think to hold fixed.

Every run is indexed, including the failures. Each cell is one row in a ledger built by reading the cell's own log rather than a receipt written afterwards, recording configuration, counts, whether the counts are exact, and the size and SHA-256 of the log they came from. This is not bookkeeping for its own sake: it recovered a valid cell whose script died between finishing the benchmark and writing its result row — a cell that is the median of its load level — and it exposed a fabricated negative sample that our own indexer had produced by mapping an older result schema positionally. A campaign that silently drops its failures is a selection of the runs that worked, which is the failure this paper is about, one level up.

The full record, stated once. The external campaign totals 223 instrumented runs in twelve named sub-campaigns: four load sweeps, a resolution sweep, a payload sweep at an incommensurate rate, a bimodality series (n=18 at one configuration), two rate-phase sweeps, the rate-independence control, a 63-cell comprehensive campaign (arms to $n \geq 5$, three new denominators, a payload×grid manipulation, a duration sweep, and a two-rate probe of the continuous value), and a distributed-mode series that never completed a run and is reported as exactly that. Six predictions for the comprehensive campaign were registered with falsifiers before it launched, and each interim reading was committed to the version-controlled record before the block that followed; three registered predictions failed and are reported beside the one that passed. From 22:02 UTC on the final night a once-per-minute log of the driver's clock discipline (residual frequency, skew, offset) runs alongside every cell, so each run joins to the clock state during it.

Verification of the manuscript itself. Every quantity this paper states from the campaign is recomputed from the committed ledger by an automated suite (2,000-plus checks, branch coverage of the analysis code at or above 95%), including every row of Tables the main text–the main text and every checkable sentence of the main text; run counts and discard totals are generated into the text from the ledger rather than typed, after an earlier draft was found quoting a count stale in nine places at once. The rendered PDF is checked separately from the source, which has caught defects the compiler passes. The suite is mutation-tested: corrupting a table value, or a value and its derived classification together, fails a check.

Moved from the main text (internal review, round 1). A validity gate refuses to report a number from a run that did not demonstrably produce output — a benchmark that fails to run discards nothing, its zero looking exactly like a measured zero; the gate has refused eight distributed attempts, refusals archived beside its acceptances. Warmup is disabled and reported with a matched sweep (retention unchanged; samples ≈ 120,000 →≈ 90,000), axes are swept one at a time, and every run, failures included, is one ledger row recording configuration, sign-separated counts, and the log's size and SHA-256. Two counting defects we found and fixed, and what the ledger's indexing exposed, are supplementary material S16.

16 S17. The transfer procedure with its incidents (moved from the main text)

The check is worth reporting only if it transfers, so we state it as a procedure rather than as a description of what we happened to do. It applies to any latency computed as a difference of timestamps taken in different processes, threads or hosts — which covers most end-to-end measurements in this literature.

(1) **Identify every span whose endpoints are stamped by different execution contexts.** In our pipeline exactly one of the three components qualifies: scheduling lag and the output interval are stamped within a single process, broker transport is not. Spans stamped within one context cannot violate causality and need no check.

(2) **Record the raw per-event values, not summaries.** The failure is invisible in aggregate — that is its defining property — so a pipeline that writes only percentiles has already discarded the evidence. This is the one requirement that may demand a change to an existing harness.

(3) **Count the fraction of events in which the span is negative**, per run and per component.

(4) **Reject by a stated rule, before looking at results.** We reject a run when any component exceeds 1% inverted or has a negative median. The threshold is a judgement and should be reported with its sensitivity (the main text); what matters more is that it is fixed in advance and applied to every condition, including those whose results look reasonable. Applying it only to suspicious results reproduces the error it exists to prevent.

(5) **Report the retention rate alongside the results**, the way a survey reports its response rate. A benchmark that discards nothing has either an unusually clean instrument or an unexamined one, and a reader cannot tell which without the number. Count *both* causes separately: a sample rejected for being negative is an instrument failure, and a sample rejected for computing to zero is a resolution failure. A single unsigned total cannot distinguish them, and we drew a wrong conclusion from one that could not (the main text).

(6) **Compare the timestamp's resolution against the latency you expect to measure, and publish both.** This is the ratio that governs the second failure mode, and neither number alone conveys it. A millisecond-grained clock on a path the experimenter believes is sub-millisecond is a stated problem, not a latent one.

(7) **Read your own reported percentiles for quantisation. This costs nothing and needs no patch.** If every percentile a benchmark prints is a whole multiple of its clock's quantum, the measurement has no resolution below that quantum and the distribution is an artefact of the grid. In the benchmark we audited, all 32 percentile values across eight runs were whole milliseconds, and three runs reported p50 = p95 = p99 = max = 1.0 — a distribution with one value in it, printed to three decimal places, in output anybody could have read. We found it only after instrumenting the source, which was unnecessary.

(8) **Put your send interval over the timestamp quantum, reduce the fraction, and look at the denominator.** A fixed interval that visits few phases within the quantum puts nearly every sample in a run at one of them, which makes the surviving fraction bimodal and irreproducible rather than merely small (in the main text). The diagnostic is arithmetic on two numbers you already know, but it is the reduced denominator q that matters, not whether the interval looks round: 500 msg/s gives $q = 1$ and spread 99.5 points, and 300 msg/s gives $q = 3$ and spread 30.5, while 889 msg/s — which sits 0.00014 away from 9/8 — gives a denominator in the hundreds and is stable to 1.7. A small q is the hazard; proximity to a simple fraction is not. The remedy is to dither the send instant, which works without having to get the arithmetic right.

(9) **Report events-per-run alongside every latency percentile.** This is the check that would have caught our second withdrawal (the main text) and the sign check would not: a median over seven events is a statement about the harness, not the system, and it looks its most convincing when the artefact is deterministic enough to repeat across a hundred runs. Any percentile computed over single-digit samples should be labelled as such.

(10) **Record the achieved rate, per run, next to the results — not the flag that was meant to produce it.** A replay harness has a nominal rate and an achieved one, and they part company silently. Ours parted company by a factor of 120 because the plans carried a compression the flag did not know about (the main text). Derive the setting from the workload rather than assuming it, verify the achieved rate against elapsed wall time before the campaign rather than after, and write the verified figure into an artefact that outlives the raw data. We had to reconstruct the rate of our own earliest corpus by arithmetic on its results, which happened to be possible and will not always be.

(11) **Before attributing an offset to every event, vary the run length and count.** Lengthen the observation window by a known factor and count how many events exceed the offset, rather than re-computing the average. A per-run cost holds that count fixed while the run grows; a per-event one grows it in proportion. The two are indistinguishable in any percentile, which is why we report the count. In our sweep the count stayed at four while events per run grew several times over (supplementary material S7).

(12) **Where retention differs between arms, bound the selection it introduces.** Unequal rejection is itself a bias, and the main text gives the worst-case imputation we use.

The cost is one pass over data the harness already produces. In our implementation the rule is under two hundred lines and runs in seconds over 2,266 runs, which is why we argue it belongs in routine practice rather than in a dedicated study.

When it matters most. Equation the main text tells a reader where to be careful without repeating our experiments: risk rises as the measured quantity approaches the instrument's own scheduling jitter, and as utilisation approaches saturation. A benchmark measuring milliseconds on a loaded machine is in the dangerous region; one measuring seconds, or running well below saturation, is not. That is a rule of thumb, and the main text says what would be needed to make it quantitative.

Moved from the main text (internal review, round 1). The check transfers to any latency computed across execution contexts — most end-to-end measurements in this literature. The load-bearing rules: (1) identify every span whose endpoints are stamped by different execution contexts, and check those; (4) reject by a rule stated before looking at results, and report its sensitivity; (5) report the retention rate beside the results, counting rejection causes separately (negative = instrument failure; computes-to-zero = resolution failure); (8) put the send interval over the timestamp quantum, reduce the fraction, and read the denominator — small q is the hazard, dithering removes it; (10) record the achieved rate, per run, verified against elapsed wall time, next to the results — not the flag meant to produce it; (12) where retention differs between arms, bound the selection. Rules (2), (3), (6), (7), (9) and (11) are stated, each with the incident that taught it, in supplementary material S17. The cost is one pass over data the harness already produces; our implementation is under two hundred lines and runs in seconds over 2,266 runs, which is why we argue it belongs in routine practice rather than in a dedicated study.

17 S18. Related work in full: variability literature and the result-by-result map (moved from the main text)

Run-to-run instability has a supporting literature, and it is on our side. The irreproducibility we measure — three-replicate medians reproducing at one load level in five, and the distribution *family* itself moving between passes of an identical configuration (the main text) — sits in a line of work showing that benchmark variability is structural, not noise. Mytkowicz et al. [6] showed that factors as innocuous as link order and environment-variable size flip published conclusions; Georges et al. [7] that JVM results without confidence intervals routinely mislead; Barrett et al. [8] that virtual machines frequently never reach the steady state their benchmarks assume, so the distribution being sampled is itself non-stationary — the closest published analogue of our regime mobility. Maricq et al. [9] found hundreds of trials necessary for stable comparisons on real hardware, and Kalibera and Jones [10] supply the replication framework. Against this stands common practice rather than any contrary finding: the surveyed norm of a handful of repetitions [11, 12] presumes a stable distribution that our phase-locked configurations demonstrably lack. We add the mechanism: at these configurations the instability is not variance around a value but a two-point support whose weights move, so no replicate count converges (the main text).

Result by result. For each headline result, the nearest support and the nearest constraint we can find:

- *Deletion law and grid structure* (in the main text). Supported mathematically by equidistribution and dither theory [13–15], and practised in profiling since randomised sampling clocks were introduced to break exactly this kind of synchronisation [16]; no empirical study reports a benchmark's retained fraction, so the nearest contradiction is implicit — every published OMB-derived latency figure [17] assumes retention near 100%, an assumption our 0.36%–100% range refutes for sub-tick paths.
- *Summary insensitivity* (the main text). Supported in spirit by Ousterhout's injunction to measure beneath the summary [18] and by reporting standards that require sample counts [12]; contradicted by no study, but silently contradicted by every report of a latency percentile without the count behind it.
- *Irreproducibility and regime mobility* (the main text, the main text). Supported by [6–9]; constrained by the same literature's remedy — more replicates — which our two-point supports show cannot converge here.
- *Scheduling-stall inversions* (in the main text). Supported by the tail-latency and scheduler literature [19–21] and by the concurrent negative-span reports [22]; qualified by Sharma et al.'s skew-threshold reading, which our zero-skew inversions show is unsafe under load.
- *The sign audit* (the main text). Ten million discarded samples, not one negative. Nothing in the literature contradicts a count; what it refuted was our own prior claim, which is the point of publishing it.

Li et al. [19] decompose the tail latency of three Linux servers into hardware, operating-system and application-level sources, and report the finding that matters most for us: measured tails are *substantially worse* than a queueing model alone predicts, with the excess traced to interference from background processes, request re-ordering under constrained concurrency, interrupt routing, power management and NUMA effects. A single-parameter queueing description is therefore a leading-order account, not a complete one — which is precisely what the main text measures directly on the instrument rather than on the system under test. Their background-interference mechanism also predicts the temporal signature we observe: a core blocked by another task delays not one event but every event arriving in that interval, so the resulting failures should appear in consecutive bursts. the main text tests that prediction and finds exactly this clustering.

Our contribution here is therefore not the observation that scheduling delay is heavy-tailed, which is established, but that this shape propagates into the *measurement* of an unrelated quantity, and that load moves the tail's weight far faster than the core's width (the main text). The practical consequence is specific: a benchmark can sit at a utilisation where its measurements look tight — the core is narrow — while the tail that actually corrupts them is already growing.

Moved from the main text (internal review, round 1). That presumption is not ours: Lozi et al. [21] document Linux scheduler bugs under which cores stay idle *for seconds* while runnable threads queue elsewhere, invisible to conventional testing because throughput stays healthy. Under exact work conservation a stamping thread would never wait while a core was free, and the geometry contrast of the table in supplementary material S25 would have to be flat; it is not, and it converges only where no core is free to migrate to.

Li et al. [19] decompose Linux tail latency into hardware, OS and application sources, report measured tails *substantially worse* than a queueing model alone predicts — precisely the excess our instrument measures — and their background-interference mechanism predicts the temporal signature we observe, consecutive corrupted events, tested and found in the main text (full engagement: supplementary material S18).

18 S19. Provenance-gap episode and the co-location withholding (moved from the main text)

18.1 A gap in our own provenance

Auditing the corpus turned the same scrutiny on our own record-keeping, and it did not survive it either: no surviving artefact records the achieved replay rate of the earliest corpus. Replay plans carry a baked-in 120× compression that the producer's flag then multiplies, so the candidate rates are 1×, 120× and 1200×, and the records that exist disagree. The rate was recovered from a physical constraint: the client's start-up cost is a wall-clock constant, so the number of events it swallows encodes the rate, and one diagnostic cell whose median reads 52.34 ms (range 51.60–53.01 across fifteen runs) — midway between the two modes at 1.6 and 103.5 ms — fixes it at four late events and hence at true real time. What the gap touches is bounded: the audit is unaffected, since causal impossibility holds at any rate; the withdrawal of the main text is stated in a form that holds at every candidate rate; and the main text's external validity is restored by the recovery. The full episode — recovery arithmetic, per-claim exposure, the disagreeing records — is supplementary material S1. The rule it produced — write the number down — is in the main text.

A manipulation that did not manipulate, and what it cost. We also withhold a co-location campaign (E-A8). The intent was to shorten T_{true} by removing the network hop — the opposite of the payload sweep, and a useful check that the T_{true} result is not an artefact of how we lengthened it. It did not work. Moving the broker onto the driver made transport *longer* at idle, 0.512 → 0.573 ms, because it traded network latency for CPU contention on the same cores; under 88% load it moved transport by 1.03×, which is no manipulation at all. The inversion rates duly failed to separate (intervals overlapping in both cells). Reporting those rates as a null would misdescribe the experiment: nothing was manipulated, so nothing was tested. We record it because the pattern is itself informative. Two of our three attempts on this axis failed for structurally similar reasons — netem delayed both paths and cancelled in the difference, co-location traded one

Table 7. Why the delay sweep is not an effect-size manipulation. Injected one-way delay at the broker, verified present at the interface, at a verified true real-time rate. TTI tracks the injection; broker transport does not move, because the same delay is added to both arrivals whose difference defines it.

Injected delay	TTI median	Transport median	Transport IQR
0 ms	3.72 ms	0.535 ms	0.162 ms
1 ms	4.67 ms	0.508 ms	0.166 ms
5 ms	8.66 ms	0.494 ms	0.159 ms
20 ms	23.61 ms	0.480 ms	0.150 ms

delay for another — which suggests T_{true} is a quantity that resists manipulation in a shared-resource system, because every route to shortening it lengthens something else. The payload sweep worked because padding adds serialisation without contending for the scheduler, and we were fortunate that it does.

19 S20. The delay-sweep withdrawal in full (moved from the main text)

We withdraw the intermediate points, and the reason is worth more than they were. An earlier version supported H1 additionally with a `netem` delay sweep, reporting a rank correlation across injected delays of 0 to 50 ms. We then re-ran that sweep at a verified real-time rate with a manipulation check, and the check failed — not because the treatment was too weak, but because it does not act on the quantity at all. Injecting delay at the broker moves end-to-end TTI almost exactly as commanded (3.72 → 23.61 ms across 0 → 20 ms) while broker transport stays flat (0.535 → 0.482 ms; Table 7).

The cause is that the delay is **common-mode**. It delays the acknowledgement travelling broker-to-producer and the record travelling broker-to-consumer by the same amount, and transport is the *difference* of stamps taken at those two arrivals, so the injected delay cancels exactly. A manipulation can be applied correctly, verified present at the interface, and still not touch the quantity under study — which is the same lesson as the rest of this paper, arriving this time in our own experimental design rather than in our instrument. The original sweep's correlation was therefore never an effect-size relationship, and we report the sweep only as the negative result it is.

20 S21. The experiment map, the claim-evidence table, and the audit threshold figure (moved from the main text)

Figure 3 is the experiment map; Table 8 is the claim-evidence table; Figure 4 shows the threshold's consequences.

21 S22. Threats and limitations in full (moved from the main text)

Threats to validity (full)

Construct validity: the check may not measure what we claim. A negative span is proof that *something* is wrong, but our attribution of it to scheduler delay in reaching the clock is an inference. Two rival explanations are available. Clock skew is ruled out directly for Testbed A, where both processes read the same operating-system clock; it is not ruled out on Testbed B, where producer and consumer may sit on different hosts, and there we rely on the co-located conditions failing at rates comparable to the distributed ones.

Coarse kernel timestamp granularity would also produce inversions, and that rival makes a different prediction we can test. Quantisation acts on each event independently, so its inversions would be scattered at random through

Campaign	Manipulated	Held fixed	What it settles
E1 concurrency	feeds *N*: 1, 9, 10, 12	rate, host, plan set	the original question: does broker choice matter?
E-B delay sweep	injected delay 0-50 ms	load, feeds	H1 effect size (direction only; confounded)
E-A / E-A3 load sweep	background load 0-12	rate, feeds, plan	H2 utilisation, H10 mixture, F_Δ recovery
E-A2 process count	processes at fixed rate	aggregate event rate	H4 oversubscription
E-C3 / E-C4 stamping	ack stamp: callback vs inline	load, in-flight, backend	H3 asymmetry (replicated)
window sweep	observation window 60-600 s	rate, host, match	start-up cost vs per-event constant
transport (x2)	feeds, powered	verified real-time rate	broker transport, equivalence + shift
E-A5/A5b/A7 stamping priority	SCHED_FIFO on the stamping threads	utilisation, to 0.003	scheduling, not utilisation (8 pairs, 7-80x)
E-A6 / E-A6b load geometry	cores free vs all duty-cycled	achieved utilisation	utilisation is not the variable (2.07x, 2.05x, at rho 0.7531)
E-A9 run-queue trace	nothing: observation only	load, priority arm	P(stall > T_{true}) predicts the rate, unfitted
E-A10 transport sweep	payload size; T_{true} 77x	load, hosts, code path	other side of the inequality; tail index 0.34
E-A8 co-location	broker on the driver	utilisation, to 0.002	nothing: transport did not move, so it is withheld
OMB external	instrumented discard counter	our broker, under load	the exposure is not ours alone

Every claim in Section 7 traces to one row. No row both generates and tests the same hypothesis, and a row that settled nothing says so.

Fig. 3. The experiment map. Each campaign manipulates one variable, holds the others fixed, and settles one claim. The delay sweep is marked confounded because injecting delay also inflates variance (the main text), so we use it for direction only.

a run; a descheduling event, by contrast, makes a *consecutive run* of events wake late together. A Wald–Wolfowitz runs test on the sequence of inversion signs separates the two: independence gives $z \approx 0$, clustering $z \ll 0$. Across conditions the inversions are strongly clustered — median $z = -6.9$ at the co-located floor, -6.8 idle, -5.1 saturated — and the clustering is present even with no background load, so it is intrinsic to the stamping path rather than induced by contention. This is the signature of a shared scheduling delay, not of independent quantisation, and the main text additionally reports the measured timer granularity on both platforms. We regard the Testbed A attribution as well supported and the Testbed B attribution as probable rather than established.

Table 8. What each claim rests on. E1 is the only corpus where retention is a live concern; the campaigns after it retain every run, so the breakdown-point argument of the main text is needed for one row only.

Claim (section)	Campaign	Runs per cell
Audit rejection rates (main text)	all corpora	2,266 runs total
H1 effect size (main text)	co-located vs network	15 per delay level
H2 utilisation shape (main text)	E-A, E-A3	25 per load level
H4 process count (main text)	E-A2	15 per level
H3 asymmetry (main text)	E-C3, replicated E-C4	10, then 15 per arm
Mixture structure (main text)	E-A3, nine levels	25 per level
Scheduling, not utilisation (main text)	E-A5, E-A5b, E-A7	25 per arm
Load geometry (main text)	E-A6, both geometries	25 per arm
Traced run-queue tail (main text)	E-A9	25 per arm
T_{true} sweep (main text)	E-A10, four payloads	25 per level
Broker transport (main text)	powered, replicated	15, then 8 per cell
Start-up withdrawal (main text)	window sweep, twice	3 per window
Historical: E1 (main text)	E1	8–35, *52.8–100% retained*

Internal validity: the audit selects. The check rejects unequally between arms, so the surviving corpus is not a random sample of the measured one. This is the most serious internal threat, and the main text addresses it with a worst-case bound rather than an argument. That bound holds only while retention exceeds one half, and our tightest cell sits 2.8 points above that line; a slightly worse campaign would have left the equivalence claim undefendable.

Internal validity: the surviving comparison is small. After rejection, per-cell samples run from 8 to 35. The N=1 cell cannot support inference and we treat it as descriptive; conclusions rest on the levels with 19 or more runs.

External validity: one workload, two systems, two platforms. We demonstrate the failure on a sparse bursty feed, two brokers and two operating systems. The model of the main text predicts it wherever cross-process spans are measured near the instrument's own jitter under load, but we have not shown that. Sharma et al. [22] report the same symptom in an unrelated domain, which is suggestive but is not our evidence.

Construct validity: a percentile is not automatically a property of the system. The 103 ms producer offset reproduced across both testbeds, four concurrency levels and 164 runs, and we took that breadth as evidence it was a property of the system. It was a property of the window (the main text). Reproducibility across runs distinguishes a deterministic effect from a noisy one; it does not distinguish a system effect from a harness effect, and we treated it as though it did. Every percentile in this paper is now reported with the events per run behind it.

Construct validity: the injected-delay sweep is not a clean effect-size manipulation. H1's quantitative slope (the main text) is estimated from a `tc netem` delay sweep, on the assumption that injecting d ms of delay simply raises T_{true} by d. It does not: on our bursty feed, a per-delivery delay builds a queue, so the measured transport *variance* climbs from 10 ms^2 at zero delay to 9,200 ms^2 at 50 ms — a clean offset would leave it unchanged. The delay sweep therefore confounds T_{true} with backlog-driven jitter, and we do not lean on its fitted slope. H1's robust evidence is the comparison it does not confound: the co-located arm, where $T_{\text{true}} \approx 0.5$ ms, fails wholesale, while the network arm, where T_{true} is tens of seconds, passes 15/15 on the same hardware (the main text). Testing the fine-grained slope cleanly needs an instrument that lengthens T_{true} without perturbing the scheduler, which netem is not. The payload sweep of the main text is that instrument by another route: padding adds serialisation and transfer time without contending for

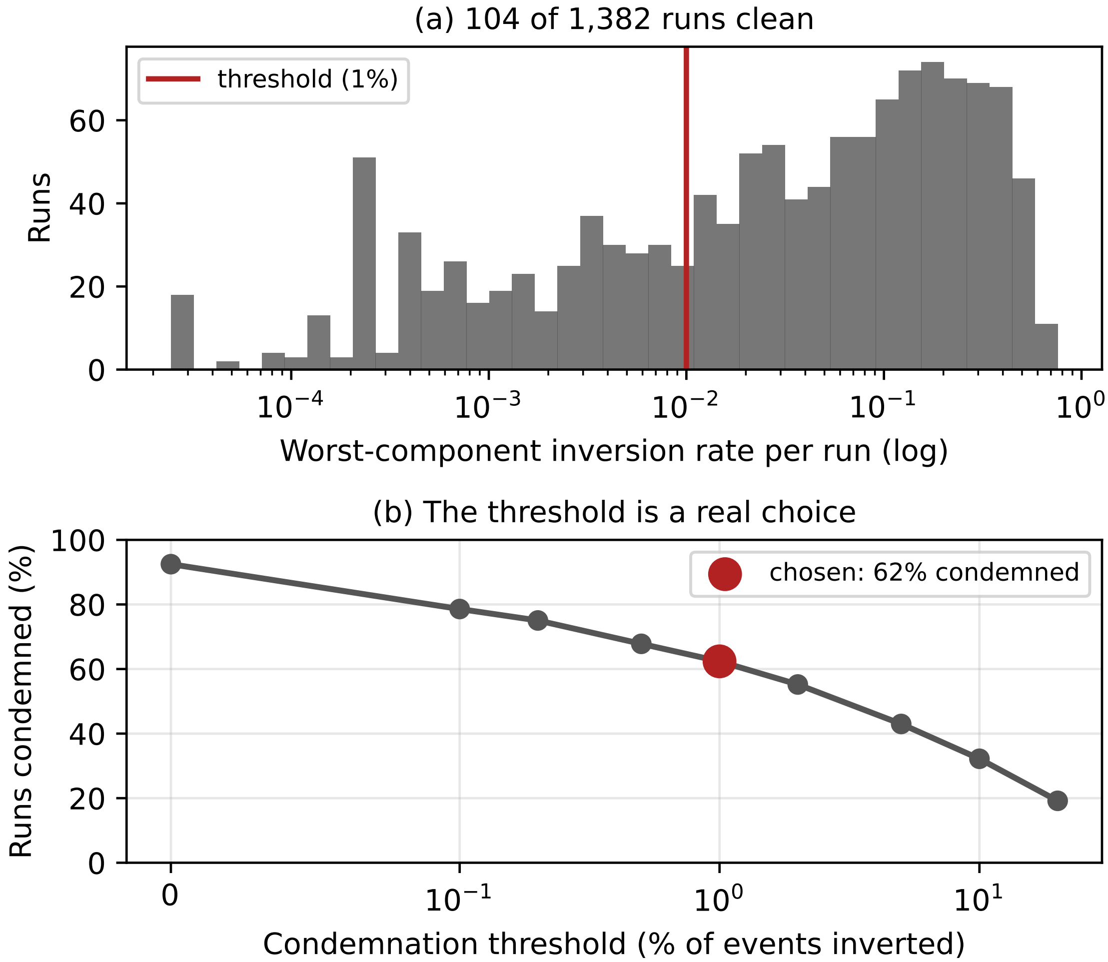

Fig. 4. The audit on Testbed A. (a) Per-run inversion rates: 104 runs are clean and the rest spread over three decades, so there is no natural threshold. (b) Share of runs rejected against the threshold. The choice matters for the run count and not for any conclusion.

the run queue, and it moves T_{true} by 77× against netem's confounded range. It is not a perfect constant offset either — serialisation pushes utilisation slightly *up*, which works against the observed direction rather than for it — but it is clean enough to carry the claim netem could not, which is why the T_{true} result rests on it and not on this sweep.

The mechanism is established; the quantity is not predicted. The clearest limit on this work is what it still cannot do. We can say what causes an inversion, manipulate it on both sides, and predict the rate to within a third in the three arms where we traced the scheduler directly. We cannot write down the rate a benchmark will suffer at a given load on a given machine. Two attempts failed and are reported above: the queueing form fits worse than the mean once the sweep reaches the range where forms diverge, and our own registered bracket missed a 1.44× growth by predicting 2.45–3.07×. Both failed for the same reason — both were parameterised in ρ, and ρ is not the variable. The tail-index rule (Equation the main text) is the nearest thing we have to a quantitative law, and it carries no load term at all: it predicts how the rate scales with T_{true}, not what the rate is. So the practical guidance this paper supports is a check to run, not a number to look up. A reader wanting to know whether their own measurement is exposed must measure it, which is precisely why we argue the check should be routine rather than the model be trusted.

Conclusion validity: multiplicity and pre-specification. Holm families are declared over the design actually executed rather than chosen afterwards, and equivalence margins were fixed before testing. The threshold in the check was fixed before the final campaign but *after* earlier campaigns had run, which is weaker than pre-registration; the sensitivity analysis of the main text exists partly to compensate.

Limitations (full)

The surviving evidence is narrower than the design. The audit removed the throughput sweep, the connection sweep above ten, the cluster arm, the durability quantification and all of Testbed A. We cannot characterise behaviour at the concurrency a multi-tenant platform would see.

E1's replay rate is inferred, not documented. the main text recovers it from the data and the recovery is, we think, correct, but it is an inference from a 52.34 ms median in one cell rather than a setting anyone recorded. A reader who does not accept the inference should read the main text as establishing that both arms met the same rate, which is all the between-system comparison needs.

The E1 reconciliation is an explanation, not a re-run of E1. Supplementary material S5 shows that restricting the powered runs to their opening seven events reproduces E1's figures in both shift and absolute level, which is why we no longer treat the two campaigns as contradictory. But it demonstrates that the window is *sufficient* to produce the disagreement, not that it is the only difference between the campaigns: E1 ran on a corpus whose replay rate we infer rather than read, and we cannot rule out that some other difference contributes as well. What we can say is that no additional cause needs to be invoked.

The mechanism is established for our client, not for Kafka in general. The blocking first `produce()` is visible in the loop trace of every run (the main text), so for this client the attribution is direct rather than inferred. Whether a different client, or a pre-created topic, or a broker configured against auto-creation would pay the same cost, we did not test. The practitioner advice in the main text is written to be safe either way.

The occupancy result is a mechanism, not a load model. the main text shows by manipulation that the inversion rate follows stamping-thread occupancy rather than utilisation, and that is the strongest causal claim in this paper. It is not a formula. We can say the rate falls fifty-fold when the stamping thread stops being preempted; we cannot say what the rate will be at a given load, and both attempts to write one — the queueing form and our own bounded bracket — failed against the sweep. A practitioner should read this as "keep the stamping thread runnable and measure what you get", not as a curve to substitute into a budget.

The external evidence is single-host, and the cross-host case is unmeasured. The 223-run campaign of the main text ran in embedded mode, so both stamps came from one JVM on one host. That is sufficient for the resolution failure, which needs no second clock — the quantum is a property of the timestamp, not of the network — but it means the cross-host channel is established by source audit and not by observation. We attempted a distributed run eight times, at three background-load levels including none at all, and the benchmark's own coordinator-to-worker protocol failed on every one; the load-starvation hypothesis we formed after the first failures is refuted by the unloaded attempts failing identically. That difficulty is worth recording rather than hiding, and it cuts both ways: a failure mode this hard to

Table 9. Retention against the producer's send interval, with message size, background load, duration and host held fixed. Only the commensurate rate is unstable, and the two incommensurate rates agree with each other despite intervals differing by 19%. Predictions were recorded before the runs.

Rate	Interval	Multiple of tick?	Retention per replicate	Spread
500/s	2.000 ms	**exact**	0.47, 1.51, 18.02, 99.99	99.5
457/s	2.188 ms	no	48.77, 49.50, 49.69, 50.87	2.1
383/s	2.611 ms	no	50.28, 51.46, 53.13	2.8

Table 10. Retention against payload at 457 msg/s, incommensurate with the millisecond grid so samples stay dephased. Implied T_{true} inverts the main text. The final column is the increment in implied latency per byte of added payload, relative to the 200 B baseline; the two smallest steps are noise-dominated and marked.

Payload	Retention (3 reps)	Median	Implied T_{true}	ΔT/byte
200 B	51.08, 52.03, 52.88	52.03%	0.520 ms	baseline
1 KB	50.11, 50.32, 51.47	50.32%	0.503 ms	$-2.600^\dagger$
2 KB	52.13, 54.62, 54.77	54.62%	0.546 ms	$1.752^\dagger$
4 KB	51.86, 54.17, 56.24	54.17%	0.542 ms	$0.686^\dagger$
8 KB	54.92, 55.22, 57.89	55.22%	0.552 ms	$0.499^\dagger$
32 KB	68.01, 68.44, 69.40	68.44%	0.684 ms	**0.630**
64 KB	84.21, 85.36, 86.82	85.36%	0.854 ms	**0.638**

† added serialisation under 0.07 ms, comparable to the 0.02–0.04 ms replicate noise, so the ratio is scatter over a small denominator. The two unmarked levels have a signal-to-noise ratio near 20 and carry the result.

instrument is one a practitioner is unlikely to find by accident, and a distributed mode that does not complete a run in repeated attempts by readers of its source is itself a fact about the state of the tooling.

We are also explicit that the clock bound on this testbed does *not* rule the channel out. `chrony` reports inter-host offsets near 0.1 ms, which is comfortably sub-tick — but the bound it places on its own error, root dispersion plus half the root delay, is 4 to 7 ms per host, so a pair may disagree by up to 12 ms against a 1 ms timestamp. An operator checking whether cross-host subtraction is safe reads the offset line and concludes it cannot happen; the figure that bounds the disagreement is two orders of magnitude larger and three lines further down the same output. The channel is therefore open on this testbed, and we could not measure it.

H3 is measured at one operating point. The symmetric-stamping comparison (the main text) was run at one in-flight request and one load level, because inline stamping requires the single in-flight setting. The direction and the one-sidedness of the effect are what the model predicts, but we do not characterise how the 0.07 ms offset scales with load or pipelining.

The five-feed acknowledgement null is unexplained, as set out in the main text.

One workload. The arrival-rate result should generalise to any sparse bursty feed, but we demonstrate it on one.

22 S23. Tables (moved from the main text; the pipeline schematic has returned to it)

23 S24. Tail recovery without a reference clock (moved from the main text)

Recovering the tail without a reference clock, and where it reproduces. Because inversion rate at a threshold *is* a quantile of Δ's tail, the tail can be read off inversion counts alone — an instrument-quality estimate needing no reference clock. The honest test of it is not internal consistency but reproduction: do the recovered tail quantiles agree between two

Table 11. H2: inversion rate against achieved CPU utilisation, no core pinning, background load swept from idle to oversubscription. The withdrawn queueing form fits at $R^2 = 0.945$ against 0.640 for a straight line.

Utilisation ρ	Inversion rate
0.003	0.007
0.253	0.009
0.504	0.022
0.628	0.047
0.753	0.132
0.878	0.207
1.000	0.208
1.000	0.221
1.000	0.264

Table 12. Bounding the unequal-retention selection effect. “Worst case” imputes every rejected Redis run at the largest value observed in that cell. Margin 1 ms.

N	Redis retained	Shift (ms)	Worst case	Equivalent
1	8/8	+0.081	+0.081	yes
9	20/27	+0.021	−0.051	yes
10	17/30	+0.116	−0.485	yes
12	19/36	+0.053	−0.227	yes

independent campaigns at matched utilisation? Below saturation they do — 12 of 17 matched quantiles overlap at 90%, and every one of the deep pre-knee quantiles does. The failures are all at $\rho = 1$, where the coordinate is degenerate — eight, ten and twelve background workers all read as full utilisation while representing different degrees of overload — so the two campaigns are not in fact at matched load there. The recovered curve reproduces wherever utilisation is a meaningful coordinate and stops when it stops being one, which is the correct boundary for the method rather than a defect in it.

24 S25. The mechanism campaigns’ tables (moved from the main text)

Table 13 is the mixture across nine load levels; Table 14 the occupancy manipulation; Table 15 the load-geometry comparison with its replication; Table 16 the payload sweep; and Table 17 the kernel-trace comparison, including the withheld arm.

25 S26. The first result set (the model and flip figures were promoted to the main text in the round-1 internal revision)

Table 18 is the first, rejected result set in full.

26 S27. The grid table (returned to the main text)

The table of replicate spread against the phase denominator q, which this section previously held, is back in the main text: the reviewer of v2.5 was right that load-bearing evidence should not sit in supplementary material. It is not duplicated here.

Table 13. The mixture, measured across nine load levels (25 runs each, pre-registered). The core width $\sigma_{\text{core}} = \text{IQR}/1.349$ grows 5× from idle to the knee; the inversion rate — the tail — grows 60×. Inversions are temporally clustered at every load (runs-test z). The three $\rho = 1$ rows are distinct oversubscription levels that the utilisation coordinate cannot separate.

Utilisation ρ	Core σ_{core} (ms)	Inversion rate	Clustering z
0.003	0.126	0.004	−6.8
0.253	0.138	0.004	−6.8
0.504	0.180	0.006	−6.8
0.628	0.181	0.024	−6.9
0.753	0.238	0.076	−5.7
0.878	0.631	0.224	−4.9
1.000	1.224	0.371	−4.3
1.000	1.293	0.317	−4.7
1.000	1.710	0.261	−4.4

Table 14. **Occupancy, not utilisation.** Stamping processes at ordinary priority versus `SCHED_FIFO`, with the background load unchanged. Utilisation is measured in both arms, not assumed: it matches to 0.001, so the manipulation moved occupancy alone. The inversion rate falls by a factor of 39–54 to the running-state floor. Utilisation-only models predict no change.

	Utilisation ρ		Inversion rate		
Load	ordinary	real-time	ordinary	real-time	factor
75%	0.7531	0.7525	0.1320	0.0034	39×
88%	0.8809	0.8812	0.3049	0.0057	54×

Table 15. **Same utilisation, different rate.** Both load geometries in one campaign. Concentrated load leaves $C - k$ cores free; spread load duty-cycles all of them and leaves none. At $k = 6$ the two arms sit at the same ρ to four decimals — in *both* campaigns — and the rates differ twofold in both. Every cell is 2 985 matched events over 25 runs; z is a two-proportion test between the arms of that row. The $k = 7$ row is where the two campaigns disagree: the original cannot separate the geometries there, the replication can, and we withdraw the stronger reading (text).

	E-A6 (original)			E-A6b (replication)			
k/C	conc.	spread	z	conc.	spread	z	ratios
5/8	0.0201	0.0379	4.09	0.0111	0.0509	8.89	1.88×, 4.61×
6/8	0.0824	0.1709	10.27	0.0600	0.1229	8.44	2.07×, 2.05×
7/8	0.2281	0.2415	1.22	0.1973	0.2342	3.46	1.06×, 1.19×

Achieved ρ, $k = 6$: 0.7531 in all four arms. $k = 5$: 0.6284/0.6250 and 0.6284/0.6259. $k = 7$: 0.8778/0.8809 and 0.8778/0.8822.

27 S28. The mechanism campaign paragraphs in full (moved from the main text)

The experiment that settles it. Move one of the two and hold the other fixed: raising the stamping processes to real-time priority makes them preempt the background load rather than queue behind it, so occupancy falls sharply while utilisation does not move — and unlike our delay injection, which cancelled in the subtraction (the main text), priority acts on the stamping threads themselves, where the model locates the failure. An occupancy mechanism predicts collapse toward the running-state floor; any account in which the rate is a function of ρ predicts no change, because ρ is unchanged by construction. The result is unambiguous (Section S25): utilisation held to 0.001 (the pre-registered manipulation check passes), and the inversion rate fell from 0.132 to 0.0034 at $\rho = 0.75$ and from 0.305 to 0.0057 at

Table 16. **A slower path is a more reliable measurement, twice.** Payload padding lengthens the true transport without touching the scheduler. Utilisation is held to a 0.012 spread; the serialisation cost pushes it slightly up, against the predicted direction. Both quantities move monotonically, in opposite directions, in both campaigns: transport spans 76.9× and 76.8×, the rate falls 4.09× and 4.26×. Each cell is 2 985 events.

	E-A10		E-A10b		
Padding (B)	Transport (ms)	Inversion	Transport (ms)	Inversion	ρ span
0	0.701	0.2603	0.761	0.2338	0.881–0.882
4 096	2.381	0.1906	2.474	0.1652	0.882–0.883
65 536	14.638	0.0888	16.059	0.0784	0.885–0.885
262 144	53.898	0.0637	58.482	0.0549	0.893–0.895

Table 17. **A kernel trace predicts the inversion rate, unfitted, in every arm that has one.** $\Pr[\text{stall} > T_{\text{true}}]$ from `sched_wakeup`/`sched_switch`, against the inversion rate measured from application timestamps in the *same* cell; the two share no instrument and no estimator. Every cell is 2 985 events. The three ordinary arms agree with their traced tails to within about a third, with no consistent sign. Every real-time arm reads exactly zero where the untraced control of the E-A9 pair reads 15/2985; that is a tracing artefact, not a floor (text). The E-A9b 88% arm is *withheld* by the instrument check — 28% drift against a 25% rule fixed in advance — and is shown because a withheld comparison is still a measurement.

Campaign	Arm	Load	ρ	Traced events	$\Pr[\text{stall} > 0.5\,\text{ms}]$ / rate	Ratio
E-A9	ordinary	88%	0.8822	551,956	0.181 / 0.231	0.78
E-A9	real-time	88%	0.8819	570,591	0.018 / 0.000	—
E-A9b	ordinary	75%	0.7543	709,819	0.168 / 0.128	1.32
E-A9b	real-time	75%	0.7531	641,308	0.016 / 0.000	—
E-A9b	ordinary	88%	0.8825	687,986	0.207 / 0.196	*1.06*†
E-A9b	real-time	88%	0.8822	649,788	0.016 / 0.000	—
untraced control (E-A9's own first attempt, probe never attached)						
—	ordinary	88%	0.8812	—	— / 0.272	—
—	real-time	88%	0.8809	—	— / 0.005	—

† withheld by the instrument check; shown, not used.

Table 18. **The first result set, later rejected.** Broker transport (median, ms) by concurrency, Testbed A, 10× replay. Every cell fails the check of the main text.

N	Kafka	Redis	Difference	p_{Holm}
1	0.999	1.006	0.007	1.0×10^{-4}
5	1.001	1.197	0.196	6.5×10^{-6}
10	1.002	1.346	0.344	9.0×10^{-11}

$\rho = 0.88$ — factors of 39 and 54, disjoint Wilson intervals, both residual rates on the floor the model names in advance, $C_0 \approx 0.004$.

Utilisation is not the variable, shown at identical utilisation. Two loads can reach the same ρ by different means: k cores flat out leaves $C - k$ genuinely free, while duty-cycling every core leaves none. If the mechanism is whether the stamping thread can find a core, the second should be worse at equal ρ — and it is (Section S25): at $k = 6$ both arms sit at $\rho = 0.7531$, identical to four decimals, same machine, same hour, and the inversion rates differ by 2.07× ($z = 10.3$

Table 19. Consistency audit of every run in the study.

Corpus	Runs	Rejected	Conditions	Usable
A (single machine)	1,382	862 (62.4%)	76	8
B (four machines)	884	459 (51.9%)	40	13
Total	2,266	1,321 (58.3%)	116	21

Table 20. H3: median broker transport under the two stamping modes, both arms at one in-flight request under matched load. `callback` is the default asymmetric instrument; `inline` stamps Kafka on the calling thread as Redis already does. The between-system gap shrinks, and the shrinkage is entirely on Kafka's side.

Stamp	Kafka (ms)	Redis (ms)	Difference (ms)
`callback` (asymmetric)	0.392	0.106	+0.286
`inline` (symmetric)	0.322	0.107	+0.215

Table 21. Injected one-way broker delay, applied identically to both systems, five feeds. Medians. The zero-delay row is rejected in both arms and marks the absence of a usable baseline.

Delay	Kafka TTI	Kafka transp.	Redis TTI	Redis transp.
0 ms		*rejected*		
5 ms	12.4 ms	5.6 ms	4,651 ms	4,645 ms
20 ms	77.8 ms	20.8 ms	31,401 ms	31,268 ms
50 ms	336.6 ms	44.9 ms	103,143 ms	102,460 ms

over 2 985 matched events per cell, disjoint Wilson intervals). A function of ρ returns one value for one input. The gap narrows at $k = 7$ (1.06×) — the mechanism showing itself: one free core out of eight is nearly as useless as none. *A full replication (E-A6b) corrects us on one point*: it reproduces the load-bearing cell almost exactly ($\rho = 0.7531$ in all four arms; 2.05× against 2.07×), reproduces $k = 5$ in direction but not size, and at $k = 7$ resolves a small difference (1.19×, $z = 3.46$) where the original could not — so we withdraw an earlier draft's reading of that convergence as a null. Both campaigns agree on the shape: twofold at $k = 6$, near-parity at $k = 7$.

The other side of the inequality, and it moves the rate the wrong way. The mechanism predicts that a *slower* path is a *more reliable* measurement, because the same stalls have a longer interval to outrun. Padding the payload lengthens T_{true} without touching the scheduler (Section S25): transport rose 76.9× while the inversion rate *fell* 4.1×, monotonically, with utilisation held to a 0.012 spread — and pushed slightly *up* by serialisation, against the observed fall, so the effect is measured against its own confound. The same numbers constrain the stall distribution sharply: a 77× larger threshold bought only a 4.1× lower rate, so the tail barely thins across two orders of magnitude — which is why mean-based counters could not see the effect and the direct trace was necessary.

28 S29. The audit, stamping-mode and injected-delay tables (moved from the main text)

Table 19 is the consistency audit of every run; Table 20 the two stamping modes; Table 21 the injected one-way delay.

Moved from the main text (internal review, round 1). Magnitudes of this size cannot be a per-event cost; the account we test is a round-trip-bound consumption pattern: our Redis consumer acknowledges each entry with `XACK` and waits

— the idiomatic tutorial pattern — so a read of two hundred entries is followed by two hundred sequential round trips, free at 0.22 ms per trip and hopeless at 20 ms, where service rate falls below arrival rate and delay is set by backlog [23]; Kafka's consumer serves from a prefetched buffer and commits on a timer, no per-record round trip to expose.

The falsification criterion was stated in advance: if amortising the acknowledgements does not help, the account is wrong. At one feed under true real-time replay, batching them in groups of two hundred takes median TTI from 4,138 ms to 103 ms — a factor of **40.2**, replicated four times — and read-loop instrumentation corrects the shape: each `XREADGROUP` returns a full batch (median 106 messages in ~32 ms), so the constraint is entirely in the acknowledgement path.

29 S31. The spread rule and the grid-membership inference in full (moved from the main text)

What the replicate spread does, and does not, predict. It is tempting to write the prediction as "spread $\approx 100/q$", and we did; that is wrong in a way that matters, because $100/q$ is the *cell width* — an upper bound, reached only when T_{true}/τ sits midway between two grid points and replicates realise both. On a grid point, one point takes nearly every run and the spread collapses. One statement covers both:

$$\text{spread} \longrightarrow \begin{cases} 100/q & \text{when } T_{\text{true}}/\tau \text{ sits mid-cell,} \\ 0 & \text{when } T_{\text{true}}/\tau \text{ sits on a grid point.} \end{cases} \tag{1}$$

The measured continuous value is $T_{\text{true}}/\tau = 0.495$, a half to within replicate noise. A half lies on every even grid (1/2, 2/4, 4/8) and never on an odd one, so at this operating point the odd denominators are predicted to show their full cell width and the even ones to show almost none. That is a prediction about specific arms before they are run, not a description after.

Every arm behaves as its position predicts. The grid table in supplementary material S27 states each rate's cell width, cell position, prediction and measured spread; the test needs no tolerance, since a spread above half the cell width separates full from flat.

All nine commensurate arms match — seven full, two flat; both classes realised, so the agreement is with a prediction that varies, not with a constant.

The denominator, not the rate that carries it. Every denominator above appears at one rate, so the pre-registered control is the 600 msg/s arm: 1.667 ms = 5/3, the same $q = 3$ as 300 msg/s at *half* the interval, recorded in advance to land near 33%. It landed at 33.97, 34.29, 34.90, 65.86 and 66.67 — three replicates on the 1/3 branch, two on the 2/3 branch, nothing between (the fifth's shutdown-hook totals arrived after our first read; the recovered 65.86 was on the upper branch). 66.67% is 2/3 to two decimals, matching the 65.35 that 300 msg/s produced on the same branch: two rates an octave apart land on the same grid because they share a denominator.

A quantity with two-point support breaks the replicate median, so the pre-specified replacement (`grid_membership_test.py`, committed with the artefact) tests each arm's mean distance to its nearest grid vertex, normalised by cell width, against a continuum null of Normal($\theta_{\text{local}}, \sigma$=1.2) replicates — both parameters measured, neither fitted to the arm under test; the p-value is one-sided by Monte Carlo.

Of twelve commensurate arms, ten are powered — the null and the grid predict separably — and 9 **of the** 10 **reject the continuum after Holm correction**, eight at the Monte-Carlo floor of 2.5×10^{-4}, including arms whose binary flat/full classification had read as smeared. The tenth (900 msg/s) rejects at 0.044 raw and at 0.131 corrected, so it does not reject: an earlier version of this document and of the main-text table counted it as a rejection while the caption claimed the correction had been applied. That is now fixed in the only durable way, by generating the table from the artefact. The two unpowered arms are the degenerate ones (400 and 800 msg/s, whose grids pass through θ), and the test reports them as undecidable rather than counting them for either side. Where every replicate lies within 3 points of a vertex the arm is branch-classifiable and the upper-branch weight carries an exact Clopper–Pearson interval in the artefact. In plain terms: judged by a test that cannot be fooled by a value with no centre, the grid is present in every arm that could show it.

Moved from the main text (internal review, round 1). A quantity with two-point support breaks the replicate median, so a pre-specified grid-membership statistic (committed with the artefact) tests each arm's distance to its nearest vertex against a measured continuum null: of twelve commensurate arms ten are powered, 9 **of the** 10 **reject the continuum after Holm correction** — eight at the Monte-Carlo floor of 2.5×10^{-4}, the tenth unresolved — and the two degenerate arms are reported as undecidable. Judged by a test that cannot be fooled by a value with no centre, the grid is present in every arm that could show it (method: supplementary material S31).

30 S32. The mechanism narrative in full (moved from the main text)

The payload-sweep fit, demoted here from the main text. Over a 77× payload span, least squares of $\log \Pr$ on $\log T_{\text{true}}$ over four payload levels gives

$$\Pr[\text{inversion}] = 0.238\ T_{\text{true}}^{-0.339}, \qquad R^2 = 0.990, \tag{2}$$

with a Student-t 95% interval on the exponent of 0.234–0.443 at two degrees of freedom. E-A10b repeats the sweep on a different day and returns 0.344 against 0.339; the prefactor drifts (0.238 against 0.216). Four points do not earn an equation in a main text, which is why this one is here.

What we withdraw about it, and why. Two claims previously made around Equation 2 do not survive, and both were withdrawn in the round-2 internal revision.

The first is the infinite-moment reading — "the exponent is below one, so the stall distribution has neither a finite mean nor a finite variance". A slope fitted to four application-level points over one decade of T_{true} does not license a statement about the moments of the underlying stall distribution, which is a different object measured over a different range. What we should have written, and now do, is the weaker and sufficient claim: over the decade that matters the survival declines so slowly that a mean-based counter is a poor instrument. That is enough to explain the observation it was invoked for — cumulative scheduler counters moving by factors of two to five against a fortyfold change in the inversion rate — without asserting that a moment fails to exist.

The second is the traced cross-check. We reported that the per-wakeup survival's windowed log–log index, 0.332 over 0.25–2 ms, was "indistinguishable" from the fitted 0.339 and therefore confirmed it independently. It is not, and it does not. Estimated properly on the same histogram of 551,956 wakeups (`scripts/tail_index_traced.py`), the exceedance index over 0.5–2 ms is 0.21 (0.20–0.21, Wilson on the nested counts) while grouped maximum likelihood over the same decade returns 1.19 (1.18–1.20, profile likelihood). Two estimators of one quantity cannot differ sixfold unless the model is wrong, and it is: the per-octave indices over that window run 1.96 and then 0.03, where a power law would give the same value twice. A bootstrap goodness-of-fit drawn from the fitted model itself agrees: $p < 0.0004$ over 2,500 replicates. The 0.332 was ordinary least squares through four nested survival points, which returns a number whether or not the points lie on a line. The agreement with 0.339 was a coincidence of window and estimator, and we should not have offered it as a confirmation.

The prediction check against the trace is unaffected and stands: the main text predicts $\Pr[\text{stall} > 0.5\,\text{ms}] = 0.301$ against the traced 0.181, over-predicting by 1.66× (1.52× on the replication), the expected direction since not every stall lands on a stamping instant. That comparison is between two probabilities, not between two fitted exponents, and it needs no power law to be meaningful.

What the histogram says instead, which is better. Asking what shape the data do have, rather than which index to quote, gives the mechanism its most direct evidence. The bucket counts are not monotone, so no power law can describe them: there are 3 local maxima. The first is the jitter core of a running thread; the second a broad hump in the hundreds of microseconds; the third is a *mode* at 2–4 ms carrying 10.5% of all wakeups and standing 4.5× above its lower neighbour. Beyond it the survival is light and ordinary: grouped maximum likelihood above 4 ms gives 2.04 (2.00–2.07), a finite variance.

The mode's position is not arbitrary. On this machine — one host, 8 online CPUs, recorded as "8 cores" in the campaign log itself and recovered independently from the geometry conditions' own k/ho rather than from a shape description – the EEVDF base slice is $0.75\,\text{ms} \times (1 + \lfloor \log_2 8 \rfloor) = 3$ ms; run-to-parity holds a woken thread off the CPU until the incumbent has exhausted that slice unless it wins the wakeup-preemption check, and with the high-resolution tick disabled the expiry is observed at the ordinary tick. A stamping thread descheduled at the wrong moment therefore reads its clock roughly a slice late. That is the second state of the main text's two-state model, observed directly, at the constant the scheduler is configured with — which is a stronger statement than any tail index would have been, and it is what replaced the withdrawn one. The constants should be read off the image under test (`base_slice_ns` and `CONFIG_HZ`) rather than assumed from the kernel version, and we report them as the explanation the mode's location supports rather than as an independently measured cause.

The confirmation is the one that manipulates the physics. Payload moves T_{true}, so it moves θ's grid position, and the main text dictates the arm's class with nothing else changed: measured $\theta(32\,\text{KB}) = 0.685$ and $\theta(64\,\text{KB}) = 0.853$ give, at $q = 3$, $\text{frac}(q\theta) = 0.055$ (on the 2/3 point: flat, pinned there) and 0.56 (mid-cell: full width back). The registered prediction was that one manipulation would move the arm onto a grid point and off it. It did (figure in supplementary material S26): at 32 KB the five replicates were 62.06, 69.17, 69.19, 69.23, 75.69 — spread 13.6, three replicates agreeing to 0.06 points — and at 64 KB they were 73.06, 83.80, 94.87, 96.83, 99.78 — spread 26.7, the upper vertex hit to a quarter point. The flat-full boundary at half the cell width (16.7) is crossed in the predicted direction by both halves. One registered detail failed: the 32 KB pin sits at 69.2, not on the vertex at 66.67 — a displacement toward θ that the next paragraph makes systematic.

The other side of the inequality, and it moves the rate the wrong way. A *slower* path should be a *more reliable* measurement, and it is: payload padding lengthened transport 76.9× while the inversion rate *fell* 4.1×, monotonically, utilisation held to a 0.012 spread and pushed slightly *up* against the observed fall (supplementary material S25). A 77× larger threshold buying only a 4.1× lower rate means the tail barely thins across two orders of magnitude — why mean-based counters could not see the effect and the direct trace was necessary.

Closing the loop: measuring the stall distribution directly. Every result so far infers the scheduler's behaviour from the inversion rate it produces. The mechanism says something stronger and directly checkable — that the inversion rate should equal the probability that a run-queue stall outlasts the interval being measured — so we measured that probability without reference to the inversions at all. Using `bpftrace` on the `sched_wakeup` and `sched_switch` tracepoints, we recorded the delay between a stamping thread becoming runnable and reaching a core, for every such transition in the run, and computed $\Pr[\text{stall} > 0.5\,\text{ms}]$ against the measured transport of that cell (supplementary material S25).

The two quantities share no instrument, no estimator and no data: one is a count of negative transports in application CSVs, the other a kernel histogram of scheduler transitions. Across three arms at two load levels they agree to within a third — ratios 0.78, 1.06, 1.32 (supplementary material S25), nothing fitted between them. The residual has no consistent sign (22% low, 6% high, 32% high), so what one arm had suggested as a directional effect was scatter; a traced scheduler quantity predicts an application-level failure rate to within a third, unfitted, and no bias is resolvable inside that.

The two-state reading accounts for the clustering (a preemption is an interval, so the events inside it are corrupted together) and predicts tails that shift vertically rather than rescale horizontally — measured close to parallel: median log-ratio spread 0.29, worst 0.91, set against the 23× failure of the scale-family collapse. We found this by looking at the data — the separability test is exploratory — so we pre-registered a confirmatory version and ran it: the campaigns below. And read as a function of load alone the model is a tautology, with no content until it is pinned on the threshold axis, where it passed: the form that restores the divided-out σ reaches $R^2 = 0.9905$ against 0.9811 (exponential) and 0.8863 (power law) at equal parameters, but freezing σ at its mean improves the fit to $R^2 = 0.9982$, so σ and p move together on this ladder and neither can be credited over the other; the commentary in full is supplementary material S12.

The experiment that settles it. Raise the stamping processes to real-time priority: occupancy falls sharply while utilisation does not move, and priority acts on the stamping threads themselves — where the model locates the failure — rather than cancelling in the subtraction as our delay injection did. The result is unambiguous (supplementary material S25): utilisation held to 0.001, and the inversion rate fell by factors of 39 and 54 at the two load levels, with disjoint Wilson intervals, both residual rates landing on the floor the model names in advance, $C_0 \approx 0.004$.

Utilisation is not the variable, shown at identical utilisation. Two loads can reach the same ρ with $C - k$ cores genuinely free or with none; at $k = 6$ both arms sit at $\rho = 0.7531$, identical to four decimals, and the inversion rates differ by 2.07× ($z = 10.3$; supplementary material S25). A function of ρ returns one value for one input. A full replication reproduces the load-bearing cell (2.05×), reproduces $k = 5$ in direction but not size, and resolves a small $k = 7$ difference (1.19×, $z = 3.46$) the original could not — so we withdraw an earlier draft's reading of that convergence as a null; both campaigns agree on the shape, twofold at $k = 6$ collapsing toward parity at $k = 7$, one free core in eight being nearly as useless as none.

31 S33. Resolution, discarded samples and sampling artefacts: the literature engagement in full (moved from the main text)

Coordinated omission is the closest neighbour, and is the mirror image. In Tene's *coordinated omission* [24] the worst samples are never taken, so percentiles are optimistic by orders of magnitude; here the samples *are* taken, the latency *is* computed, and a downstream guard rejects the result — one loses the slow tail, the other the fast bulk. Both end with a confident summary over a biased subset, and a correction for one does nothing about the other.

Not hypothetical: the benchmark we instrument records into an `HdrHistogram` [25], which ships explicit coordinated-omission machinery and none for this failure — its sound rejection of negative values is precisely what motivates the guard whose `else` branch silently deletes the sub-tick samples: a protection against one measurement artefact created the conditions for another.

The mathematics is a century old; the interaction is unreported. Weyl's equidistribution theorem [13] and the three-distance theorem [14] govern the irrational and rational regimes of Δ/τ; Schuchman's condition [15], generalised by Wannamaker et al. [26], is the remedy that the main text states as dithering. The timing literature treats a coarse tick as staleness, not deletion. What we claim as new, narrowly: not quantisation — elementary, and Cloudprofiler [27] exists because milliseconds are known to be too coarse — but the *interaction* of quantised stamp, few-phase producer and positivity guard, producing a retention fraction bimodal, irreproducible between identical runs, invisible in the reported summary, its damage set by the denominator of the interval-to-quantum ratio in lowest terms rather than by "round" rates.

Run-to-run instability has a supporting literature, and it is on our side. Mytkowicz et al. [6], Georges et al. [7], Barrett et al. [8] and Maricq et al. [9] establish that benchmark variability is structural, that factors as innocuous as link order flip published conclusions, and that hundreds of trials can be needed — against the surveyed norm of a handful of repetitions [11, 12]. Our phase-locked configurations supply the mechanism those studies lack: instability that is not variance around a value but a two-point support whose weights move, so no replicate count converges (the main text).

31.1 S33.1. The payload sweep's underpowered rows, and the comprehensive campaign's duration and plateau paragraphs (moved from the main text)

The sweep discriminates only where the manipulation exceeds the noise, and we say which levels those are. Replicate noise in the implied latency is 0.02–0.04 ms; at 32 and 64 KB the added serialisation (0.26, 0.52 ms) exceeds it near twentyfold and the per-byte estimates agree, while below 8 KB it is comparable to the noise and the ratios scatter (−2.600, 1.752, 0.686, 0.499) — rows reported rather than dropped, because omitting them would make the table look more decisive than the experiment was. The law rests on the two largest levels; our first attempt sat entirely inside the underpowered regime and settled nothing, so the sweep was extended rather than reported.

Duration does nothing, which kills the natural alternative. A cumulative drift walk would scale with run length — registered: intermediates rise with duration. Measured over one, three and ten minutes at 500 msg/s: intermediate counts 1, 1, 0, branch values pinned to the vertices throughout (0.43, 1.11, 100.00 at ten minutes), probability at most $(1/3)^3 \approx 0.04$ under the walk's own reading. The walk is falsified; what stands is per-send jitter, applied at each send, not accumulated.

The continuous value plateaus, and one miss stands. Incommensurate arms at 1053 and 1219 msg/s measured $\theta = 47.2\%$ and 48.2% — the downward trend below 889 msg/s does not continue, refuting the linear extrapolation one registered prediction relied on — and the 1250 msg/s arm, predicted mid-cell and full, instead pinned on the 2/5 vertex (40.66–42.00, one at 48.26): probability ≈ 0.08 under the branch law, recorded as a miss — yet that arm sits six to seven points below the continuous value beside it, which no continuous account produces: the grid is present even where the branch-probability law under-delivers. The payload sweep succeeded where co-location failed because padding adds serialisation without contending for the scheduler.

References

[1] Apache Kafka, "KIP-526: Reduce producer metadata lookups for large number of topics," Apache Software Foundation, Kafka Improvement Proposal, 2020, documents the metadata fetch a producer performs before its first send to an uncached topic.

[2] StatsBomb, "Statsbomb open data," *GitHub Repository*, 2023. [Online]. Available: https://github.com/statsbomb/open-data

[3] H. Liu, W. Hopkins, A. M. Gomez, and J. S. Molinuevo, “Inter-operator reliability of live football match statistics from opta sportsdata,” *International Journal of Performance Analysis in Sport*, vol. 13, no. 3, pp. 803–821, 2013.

[4] L. Pappalardo, P. Cintia, A. Rossi, E. Massucco, P. Ferragina, D. Pedreschi, and F. Giannotti, “A public data set of spatio-temporal match events in soccer competitions,” *Scientific Data*, vol. 6, no. 1, p. 236, 2019.

[5] M. Mohammad, “Analysis of design patterns and benchmark practices in Apache Kafka event-streaming systems,” *arXiv preprint arXiv:2512.16146*, 2025.

[6] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, “Producing wrong data without doing anything obviously wrong!” in *Proc. 14th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS).* ACM, 2009, pp. 265–276.

[7] A. Georges, D. Buytaert, and L. Eeckhout, “Statistically rigorous Java performance evaluation,” in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages and Applications (OOPSLA).* ACM, 2007, pp. 57–76.

[8] E. Barrett, C. F. Bolz-Tereick, R. Killick, S. Mount, and L. Tratt, “Virtual machine warmup blows hot and cold,” *Proceedings of the ACM on Programming Languages*, vol. 1, no. OOPSLA, pp. 52:1–52:27, 2017.

[9] A. Maricq, D. Duplyakin, I. Jimenez, C. Maltzahn, R. Stutsman, and R. Ricci, “Taming performance variability,” in *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI).* USENIX Association, 2018, pp. 409–425.

[10] T. Kalibera and R. Jones, “Rigorous benchmarking in reasonable time,” in *Proceedings of the 2013 International Symposium on Memory Management (ISMM).* ACM, 2013, pp. 63–74.

[11] E. van der Kouwe, G. Heiser, D. Andriesse, H. Bos, and C. Giuffrida, “SoK: Benchmarking flaws in systems security,” in *Proc. 4th IEEE EuroS&P.* IEEE, 2019, pp. 310–325.

[12] T. Hoefler and R. Belli, “Scientific benchmarking of parallel computing systems,” in *Proc. ACM/IEEE SC.* ACM, 2015, pp. 73:1–73:12.

[13] H. Weyl, “Über die gleichverteilung von zahlen mod. eins,” *Mathematische Annalen*, vol. 77, pp. 313–352, 1916.

[14] V. T. Sós, “On the distribution mod 1 of the sequence $n\alpha$,” *Annales Universitatis Scientiarum Budapestinensis, Sectio Mathematica*, vol. 1, pp. 127–134, 1958.

[15] L. Schuchman, “Dither signals and their effect on quantization noise,” *IEEE Transactions on Communication Technology*, vol. 12, no. 4, pp. 162–165, 1964.

[16] S. McCanne and C. Torek, “A randomized sampling clock for CPU utilization estimation and code profiling,” in *Proc. USENIX Winter Technical Conf.*, 1993, pp. 387–394.

[17] OpenMessaging Project, Linux Foundation, “The OpenMessaging benchmark framework,” 2018, available at openmessaging.cloud/docs/benchmarks/; accessed 19 August 2026.

[18] J. Ousterhout, “Always measure one level deeper,” *Communications of the ACM*, vol. 61, no. 7, pp. 74–83, 2018.

[19] J. Li, N. K. Sharma, D. R. K. Ports, and S. D. Gribble, “Tales of the tail: Hardware, OS, and application-level sources of tail latency,” in *Proc. ACM SoCC.* ACM, 2014.

[20] J. Dean and L. A. Barroso, “The tail at scale,” *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013.

[21] J.-P. Lozi, B. Lepers, J. Funston, F. Gaud, V. Quéma, and A. Fedorova, “The Linux scheduler: a decade of wasted cores,” in *Proc. 11th European Conf. Computer Systems (EuroSys).* ACM, 2016, pp. 1–16.

[22] A. Sharma, D. Shah, D. Lariviere, and H. ElBakoury, “Time, causality, and observability failures in distributed AI inference systems,” *arXiv preprint arXiv:2604.21361*, Apr. 2026, open Compute Project, Unified Intelligent Infrastructure workstream.

[23] L. Kleinrock, *Queueing Systems: Volume I - Theory.* Wiley-Interscience, 1975.

[24] G. Tene, “How NOT to measure latency,” 2015, talk, Strange Loop / QCon; coined “coordinated omission”.

[25] ——, “HdrHistogram: A high dynamic range histogram,” 2015, used by the OpenMessaging Benchmark; available at hdrhistogram.org.

[26] R. A. Wannamaker, S. P. Lipshitz, J. Vanderkooy, and J. N. Wright, “A theory of nonsubtractive dither,” *IEEE Transactions on Signal Processing*, vol. 48, no. 2, pp. 499–516, 2000.

[27] S. Yang, J. Jeong, B. Scholz, and B. Burgstaller, “Cloudprofiler: TSC-based inter-node profiling and high-throughput data ingestion for cloud streaming workloads,” *arXiv preprint arXiv:2205.09325*, 2022.

[28] EMQ Technologies, “`emqtt-bench`: MQTT benchmark tool — `src/emqtt_bench.erl`,” Source repository, 2026, https://github.com/emqx/emqtt-bench; a millisecond cross-process difference recorded under the guard `E2ELatency > 0` with no counter for the rejected samples. Read 2026-08-20.

[29] J. Axboe, “fio: Flexible I/O tester — `gettime.c`,” Source repository, 2026, https://github.com/axboe/fio; `ntime_since`, `utime_since` and `mtime_since_tv` each return zero for a negative interval, under the comment “time warp bug on some kernels?”. Read 2026-08-20.

[30] A. D. Brunelle, “`blktrace/btt` — `btt/inlines.h`,” Source repository, 2026, `tdelta()` returns one nanosecond when the stage timestamps are out of order: `return (from < to) ? (to - from) : 1;`. Read 2026-08-20.

[31] IOP Systems, “Rezolus: systems performance telemetry — `scheduler_discarded_samples`,” Source repository, 2026, https://github.com/iopsystems/rezolus; the metric counts samples discarded “because the two timestamps arrived out of order across CPUs”. Read 2026-08-20.

[32] Network-Based Computing Laboratory, Ohio State University, “OSU micro-benchmarks 7.5 — `osu_latency`,” https://mvapich.cse.ohio-state.edu/benchmarks/, 2025, `c/mpi/pt2pt/standard/osu_latency.c` divides the round trip by two; the documentation reports “average one-way latency”. Read 2026-08-20.

[33] D. Turner and X. Chen, “NetPIPE: A network protocol independent performance evaluator,” https://netpipe.cs.ksu.edu/, 2025, “Latencies are calculated by dividing the round trip time in half for small messages”. Read 2026-08-20.

[34] Intel Corporation, "Intel MPI benchmarks — `PingPong`," Intel MPI Library User Guide, 2021, "Reported timings — time = $\Delta t/2$"; the companion `PingPing` reports the unhalved time. Read 2026-08-20.

[35] S. Hunold and A. Carpen-Amarie, "Reproducible MPI benchmarking is still not as easy as you think," *IEEE Transactions on Parallel and Distributed Systems*, vol. 27, no. 12, pp. 3617–3630, 2016.

[36] Confluent, "`librdkafka` introduction — delivery reports," https://github.com/confluentinc/librdkafka, 2026, the delivery-report callback `dr_msg_cb` is served by `rd_kafka_poll()` and other queue-serving calls, so it runs on the application's thread rather than asynchronously. Read 2026-08-20.

[37] J. Axboe, "`fio` HOWTO — interpreting the output," https://fio.readthedocs.io/, 2026, for asynchronous ioengines completion latency runs "to when the I/O's completion was reaped by fio". Read 2026-08-20.

[38] Advanced Micro Devices, "HIP runtime API: Event management," https://rocm.docs.amd.com/projects/HIP/en/latest/, 2026, under `hipEventElapsedTime()`: "the time that the event recorded may be significantly after the host calls hipEventRecord()". Read 2026-08-20.

[39] NVIDIA Corporation, "CUDA C++ best practices guide, using CUDA GPU timers," https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/, 2026, section 9.1.2: "The device will record a timestamp for the event when it reaches that event in the stream." Read 2026-08-20.

[40] Mellanox Technologies, "`sockperf`: network benchmarking utility," https://github.com/Mellanox/sockperf, 2026, the run summary prints `SentMessages`, `ReceivedMessages` and `SkippedMessages` beside the latency percentiles, and a dropped/duplicated/out-of-order line derived from sequence numbers. Read 2026-08-20.

[41] OpenTelemetry Authors, "OpenTelemetry metrics data model — exponential histogram: Zero count and zero threshold," https://opentelemetry.io/docs/specs/otel/metrics/data-model/, 2026, `zero_count` holds values at or below a zero threshold, including "values that have been rounded to zero"; `count` is defined to include it. Read 2026-08-20.

[42] EEMBC, "CoreMark-PRO run rules," https://github.com/eembc/coremark-pro, 2026, "Each workload must run for at least 1000 times the minimum timer resolution." Plain CoreMark instead sets an absolute ten-second floor. Read 2026-08-20.

[43] Standard Performance Evaluation Corporation, "SPEC power and performance benchmark methodology, version 2.3," https://www.spec.org/power/docs/SPEC-Power_and_Performance_Methodology.pdf, 2025, clause 8.2.2, Power Reporting: the benchmark "should report the number of samples that may have a return value 'Unknown"' and those exceeding the maximum allowed uncertainty. Read 2026-08-20.

[44] ——, "SPECjms2007 run and reporting rules, version 1.0," https://www.spec.org/jms2007/docs/RunRules.html, 2014, clause 3.6.2 records delivery time as a histogram of default granularity 100 ms; the only timing-quality requirement is clause 3.2's 100 ms bound on inter-node clock agreement. Read 2026-08-20.

[45] K. Sachs, S. Kounev, J. Bacon, and A. P. Buchmann, "Performance evaluation of message-oriented middleware using the SPECjms2007 benchmark," *Performance Evaluation*, vol. 66, no. 8, pp. 410–434, 2009.

[46] Joint Committee for Guides in Metrology, "Evaluation of measurement data — guide to the expression of uncertainty in measurement (JCGM 100:2008)," https://www.bipm.org/documents/20126/2071204/JCGM_100_2008_E.pdf, 2008, clause F.2.2.1, "The resolution of a digital indication". Read 2026-08-20.

[47] J. W. Pratt, "Remarks on zeros and ties in the Wilcoxon signed rank procedures," *Journal of the American Statistical Association*, vol. 54, no. 287, pp. 655–667, 1959.

32 S34. Referee-round sensitivity artefacts (internal review, round 1)

Three artefacts added in the round-1 internal revision, each recomputable by a committed script. Table 22 applies the audit gate to the powered transport campaigns and bounds the selection the gate introduces (`powered_gate_sensitivity.py`); the condition-level threshold sweep behind the main text's Section 6.2 sentence is `first_result_threshold_sweep.csv` (no first-result cell fully usable at any threshold up to 20%; best cell 23/30); the traced survival slopes that used to back the main text's effective-exponent statement are `traced_tail_slope.csv` (windowed log–log index 0.332 over 0.25–2 ms, 4.128 over 4–32 ms). That last artefact is retained for the record but is *superseded*: the slopes in it are least-squares fits through nested survival points with no interval, and S32 explains why they do not estimate what we claimed they estimated. The estimators that replace them are in `scripts/tail_index_traced.py`.

33 S35. How these results were reached: the chronology, and everything withdrawn

The main text is organised around what the evidence supports. This section holds what that reorganisation displaced: the order in which the work happened, every claim withdrawn, and the pre-registrations that failed. It is kept in full because an audit's credibility rests on its own failures being visible, and because a reader checking whether the corrections were real should not have to take the summary on trust.

Table 22. The audit gate applied to the powered transport campaigns. "Flip V^*" is the smallest value at which imputing every condemned Redis run at V^* would flip the Hodges–Lehmann shift's sign; the condemned runs' observed medians centre on 0.10–0.12 ms.

Campaign	N	Redis ret. (%)	HL all	HL gated	Δ	Flip V^*
powered	1	53.3	0.4095	0.4084	−0.0011	—
powered	9	43.7	0.4181	0.4156	−0.0025	0.60
powered	12	37.0	0.4197	0.4170	−0.0027	0.60
replication	1	25.0	0.3969	0.3807	−0.0162	0.55
replication	9	45.8	0.4120	0.4137	+0.0017	0.60
replication	12	39.8	0.4132	0.4157	+0.0025	0.55

33.1 S35.1. The answer we obtained first, and why it was wrong

The study began as a broker comparison on a live sports feed. The first complete result set was monotone in concurrency, significant after correction ($p = 9.0 \times 10^{-11}$), and had a tidy architectural story: a twentyfold end-to-end gap between the two brokers. It passed replicate counts, effect sizes, corrected significance and sanity plots.

Two independent defects destroyed it. The sign check of the main text's consistency check rejected 862 of the 1,382 runs on that testbed, including every run behind the headline. Separately, the twentyfold gap itself was a per-run start-up cost read as a per-event constant: the median was taken over as few as seven events per run, and a deterministic per-run artefact reproduces across a hundred runs beautifully. The second defect is the more instructive one, because the run data were causally consistent throughout. Causal consistency is necessary, not sufficient.

33.2 S35.2. The claim about the benchmark that we withdrew

An earlier version of this work read the OpenMessaging Benchmark's discards as causality violations. They are not. The instrumented counter we had added could not distinguish sign, and separating it showed the Kafka-driver corpus's 10,913,263 discarded samples contain not one negative: they are resolution failures, samples whose true delivery was faster than the timestamp quantum. The refuting evidence had been in our own artefact from the day we committed it, in a field we had printed and not read. The same separated channel then caught 41,403 genuine negatives in the Redis-driver replication, which is the result the main text reports.

We also called the mechanism behind those Redis-driver negatives a *candidate* — a sub-millisecond skew between two views of a virtualised clock — and hedged it as such through the round-1 internal draft. The hedge was unnecessary, and reading the driver removes it. `RedisBenchmarkConsumer.java` takes the publish timestamp from `entry.getID().getTime()`, the stream-entry identifier the Redis *server* assigns on `XADD`, while the Kafka driver uses a producer-set stamp. The Redis span is therefore a difference between two millisecond-floored clocks on two hosts, for which $-1000\,\mu$s is the only negative value expressible; the Kafka span floors two stamps differenced inside one JVM, and never goes negative. The corpus matches both predictions exactly. Hedging a claim the source code settles is its own kind of imprecision, and the main text now states the mechanism.

33.3 S35.3. The causality framing, withdrawn in this version

Through version 2 this paper described its first failure mode as a causality violation, on the argument that an acknowledgement causally precedes a receipt. That argument is wrong, and a reader pointed it out. The broker's append precedes both the producer's receipt of the acknowledgement and the consumer's receipt of the record; those are two branches of one cause, and neither precedes the other. A negative difference between them is therefore possible without any physical impossibility, and is in fact routine when the producer's callback thread is descheduled.

We settled the question on the corpus rather than by argument. Every archived run that joins producer and consumer events was recounted, span by span (`scripts/recount_spans.py`, output committed as `docs/results/span_recount.csv`). Over 738,730 events in 5,913 runs, the acknowledgement-referenced span is negative 62,264 times and the send-referenced span is negative 0 times. Nothing in the corpus arrived before it was sent.

What changed as a result: the framing, the justification of the gate, and the claim that the failure is self-detecting. What did not change: the rejection counts, which are unaltered as numbers and now described as reference-stamp failures; and every manipulation result in the main text's mechanism section, which rests on moving scheduling and observing the rate move, not on what the negatives are called.

33.4 S35.4. The M/G/1 form, withdrawn

We adopted an M/G/1 waiting-time framing as a leading-order account of the stall, following its use for benchmark bias elsewhere, and pre-registered it as a hypothesis with a falsifier. Extended to the region where the candidate forms diverge ($\rho = 0.881$–0.990), the M/G/1 form fitted *worse than the mean*: $R^2 = -0.05$, against a fitted exponential's 0.93. Our own registered bracket also missed, predicting 2.45–3.07× growth against a measured 1.44×. Both candidates were parameterised in ρ, and the real-time-priority experiment shows ρ is not the variable. The Pollaczek–Khinchine result remains the motivation for expecting a heavy waiting tail; it is not a fitted result here.

33.5 S35.5. The traced tail-index cross-check, withdrawn

The round-2 internal review asked for the traced tail index to be estimated with an interval rather than eyeballed from a slope. Doing so refuted the cross-check it was meant to strengthen, and the claim is withdrawn; S32 gives the numbers and the argument. In summary: two proper estimators on the same histogram disagree sixfold because the survival is not a power law over the window we quoted, and the agreement we reported between the traced index and the payload-sweep exponent was a coincidence of window and estimator. The payload-sweep fit itself is unchanged and is now reported in S32 rather than in the main text, since four points do not earn an equation there. The mechanism results are untouched: none of them rests on the value of an exponent, only on the direction the rate moves when T_{true} moves, which is measured and monotone.

This is the third occasion on which a number in this work survived internal review and failed the first time it was estimated with an interval attached. We record that pattern because it is the argument of the paper turned on its author: a statistic that carries no uncertainty is not yet a measurement.

33.6 S35.6. Pre-registration outcomes

Six predictions were registered with falsifiers before the comprehensive campaign. One was confirmed (the payload flip, the main text's payload-flip figure); three falsifiers fired; one was not evaluable once smeared arms appeared, because the branch-count statistic it relied on has no value on a continuum; and one split. The jitter-kernel conjecture advanced after that campaign was killed by a subsequent alternation test. The plateau observation at 1053 and 1219 msg/s refuted the linear extrapolation that one registered prediction had assumed.

33.7 S35.7. A gap in our own provenance

For part of the campaign the achieved send rate was recorded from the configuration flag rather than measured against elapsed wall time, and for a subset of runs the two disagreed. The affected conditions were re-derived from per-event timestamps and the ledger corrected; the incident is the reason rule (10) of the main text's checklist exists in the form it does. It is also the reason the main text reports measured pacer jitter rather than a configured rate.

33.8 S35.8. The distributed-mode gap

The OpenMessaging Benchmark's distributed mode failed to complete a run on every attempt we made, across two release versions. The cross-host exposure of that particular tool therefore rests on reading its source together with our own two-host harness, not on running the tool itself cross-host. Diagnostics are in the upstream issue draft accompanying this artefact.

34 S36. The audited-harness registry, in full

The main text says how many independent tools dispose of an inverted or sub-quantum sample without counting the disposal. This section is the evidence behind that count. Each row of `docs/results/external/harness_registry.csv` carries one source line, the file and symbol it came from, the upstream URL and the date it was read. The class labels are not stored in the file: they are recomputed on every build, and on every test run, by the same classifier that read the OpenMessaging Benchmark a month before the other tools were audited. A reader who disagrees with our reading of a guard can therefore check the line against the pattern that matched it rather than against our prose. Table 23 is the fold of those rows up to one verdict per tool.

Table 23. Audited harnesses. *Filter* admits only positive samples; *substitute* replaces an inverted interval with a constant; *counts* means the disposal is exposed in the output.

Tool	Language	Disposal	Counts	Source
OpenMessaging Bench.	Java	filter	no	[17]
`emqtt-bench`	Erlang	filter	no	[28]
`fio`	C	substitute 0	no	[29]
`blktrace/btt`	C	substitute 1 ns	no	[30]
Rezolus	Rust	—	yes	[31]

Two details are worth stating plainly. First, `fio`'s guard appears in three separate timing functions, each returning zero for a negative interval, and the comment above it — "time warp bug on some kernels?" — is a question rather than a diagnosis. The behaviour is defensive and reasonable in isolation; what is missing is any record that it fired. Second, `btt`'s `tdelta()` returns one nanosecond whenever the stage timestamps are out of order, with no comment, no counter and no mention in the manual page. A substitution of this kind is harder to detect than a filter, because the sample count is unchanged: any retention rate computed downstream reads 100%.

Two patterns had to be added to the classifier before it could see these tools, and both additions are recorded here because they bound what the survey can claim. The original patterns were written against JVM sources and could not match Erlang's guard, which is a short-circuit rather than an `if`, nor Erlang's clock read, which carries its unit as an argument. The survey therefore reports what the classifier can currently recognise, which is a lower bound on the field rather than a census of it.

35 S37. Two neighbouring literatures, and what they do about the second clock

Two items raised in internal review belong here rather than in the main text, which sits at the journal's twelve-page limit and at its cap of forty-five references. Both are relevant to Section VII-B, and neither changes a result.

35.1 S37.1. "One-way latency" that is half a round trip

The most common way to avoid the second clock is to use only the first, by measuring a round trip and halving it. Three widely used benchmarks do this, and we checked each against its own source and documentation rather than against custom:

- **OSU Micro-Benchmarks** [32]. `osu_latency.c` computes `(t_total * 1e6) / (2.0 * iterations)`, and the documentation reports "average one-way latency". The halving is visible only in the source.
- **NetPIPE** [33]. The documentation is explicit — "[l]atencies are calculated by dividing the round trip time in half for small messages" — and the source comment asserts that the halved value "is now the 1-directional transmission time".
- **Intel MPI Benchmarks** [34]. The user guide states "[r]eported timings — time = $\Delta t/2$" for `PingPong`, against `PingPing`, which reports the unhalved time. The guide does not call the result a one-way latency.

The three differ in what they disclose — one claims one-way without saying it halves, one says both, one says it halves without claiming one-way — but none of them states the assumption the halving requires, which is that the two directions are equal. That assumption is a strong one on a cloud path, and it is the assumption our own transport proxy declines to make (Section III-C). We raise it not as a defect in those tools, whose paths are typically symmetric by construction, but because halving is the standard alternative to the subtraction this paper is about, and its cost is unstated wherever we looked.

35.2 S37.2. Drift-aware synchronisation in MPI benchmarking

Hunold and Carpen-Amarie give the most careful treatment of clocks in the benchmarking literature we found [35]. They criticise earlier MPI benchmarks for correcting the clock offset only, when node clocks also drift, and adopt a drift-aware synchronisation that fits a linear model per process; the duration of a collective is then the difference between the maximum finishing time and the minimum starting time over all processes, taken against that corrected global clock.

The contrast with our setting is the useful part. They remove the second clock by building a better one, and then subtract global timestamps as though a single clock had produced them. That is the right move for their measurement, and it is unavailable for ours in two ways. Their correction is fitted offline over a whole run, whereas a benchmark reporting a latency per message has no such luxury; and a fitted global clock still leaves a residual offset on any individual interval, for which they give no error model, because they never consider the case where such a difference comes out negative. The word does not appear in their manuscript. Our reading is that this is not an oversight but a consequence of scale: at the millisecond durations of a collective over hundreds of processes, a residual of a few microseconds cannot invert the sign, and the question does not arise. It arises here because the interval is smaller than the instrument.

36 S38. Where the stamp is written, in other runtimes

Section V argues that an acknowledgement stamp is late because the thread that writes it must first be scheduled. That is not peculiar to a JVM benchmark, and three widely used runtimes document the same structure in their own terms. None of them frames it as a hazard, which is the point: the property is public, and the consequence for a cross-process subtraction is not drawn anywhere we could find.

The stamp is taken when the application next asks for it. `librdkafka`, the client under the benchmark's Kafka driver, serves its delivery-report callback from `rd_kafka_poll()` and the other queue-serving calls [36]. The callback therefore runs on the application's own thread, at whatever moment that thread next polls, rather than when the broker's acknowledgement arrives. Our δ_{ack} is exactly the interval between those two events.

The definition itself names the reap, not the completion. `fio` defines completion latency, for asynchronous ioengines, as running to "the time …when the I/O's completion was reaped by fio" [37]. The definition is honest and the scope matters: for synchronous I/O the same documentation says only "to when it was completed". For the asynchronous case — the one used for latency work — a reaping stamp is written into the definition of the metric.

Vendors document the device-side stamp; one draws the consequence. AMD states, under `hipEventElapsedTime()`, that events "record their timestamp when they reach the head of the specified stream" and that "the time that the event recorded [sic] may be significantly after the host calls hipEventRecord()" [38]. NVIDIA documents the same mechanism — "[t]he device will record a timestamp for the event when it reaches that event in the stream" [39] — without drawing AMD's conclusion. We note the asymmetry rather than flatten it: both vendors describe a stamp written at a moment the host does not choose, and only one says out loud how far from the host call it can be.

37 S39. Neighbouring rules, and what they already require

Two of this paper's recommendations have partial precedents, and the honest thing is to say where. Neither displaces the contribution, because neither is applied where a streaming benchmark reports a latency; but a referee is entitled to see them named.

Counting what was not measured. `sockperf` prints `SentMessages`, `ReceivedMessages` and `SkippedMessages` in the same summary block as its latency percentiles, together with a dropped/duplicated/out-of-order line derived from sequence numbers [40]. The counters are error counters rather than a retention rate, but the reader of that summary can see the denominator the percentiles were computed over, which is what Section **??** asks for.

The OpenTelemetry exponential-histogram format goes one step further and reserves a field for it. `zero_count` holds observations at or below a zero threshold, described in the specification as including "values that have been rounded to zero", and the total `count` is defined to include that bucket [41]. We do not claim it as a general sub-resolution indicator, because it is not one: it captures unrepresentable mass only at the bottom of the scale. What it establishes is narrower and still useful — that a mainstream telemetry format already treats rounded-to-zero observations as data to be carried beside the total rather than dropped, which is precisely what the guard of Section **??** does not do.

38 S40. What standardised benchmarks require about resolution

Section **??** asks that a harness state its timestamp resolution and count what it discards. Standardised benchmarking already contains rules of that form, and one of them is almost exactly the rule we propose. Setting them beside the flagship standardised messaging benchmark is the sharpest way to show the gap, so we do that here rather than assert it.

A rule of the right form, in embedded benchmarking. EEMBC's CoreMark-PRO run rules require that "[e]ach workload must run for at least 1000 times the minimum timer resolution" [42]. That is the form of rule this paper argues for: a floor on the measured interval expressed *relative to the instrument*, not in absolute time. The distinction is easy to lose, and worth keeping — plain CoreMark sets an absolute ten-second floor instead, with no reference to resolution at all, and the two rules are not interchangeable.

A rule about unusable samples, in power measurement. SPEC's Power and Performance Methodology, clause 8.2.2, says the benchmark "should report the number of samples that may have a return value 'Unknown"' and the number whose uncertainty exceeds the maximum allowed [43]. Two qualifications matter and we state them rather than round them off: the modal verb is *should*, in a methodology document rather than run rules, and the samples in question are unusable readings rather than samples excluded after the fact. Within those limits it is the same instinct — publish the count of what the instrument could not measure.

And none of it, in standardised messaging. SPECjms2007 is the flagship standardised benchmark for message-oriented middleware. Its run rules record delivery time as a per-destination histogram whose default granularity is 100 ms and whose default upper bound is 10 s [44, 45]. Searching the run rules, design document and user's guide for a resolution clause, a minimum measurable interval, or a counter of discarded latency samples returns nothing: the only timing-quality requirement anywhere in them is clause 3.2's bound of 100 ms on *inter-node clock agreement*, which is a skew limit and not a resolution limit. The reported `deliveryTime` element carries an average, a maximum, a granularity and a ninetieth percentile, and no field for samples outside the histogram's range.

We are careful about one thing here. The user's guide does document sample discarding elsewhere — `trimInitialSamples` and `trimFinalSamples`, "this many samples are thrown away" — but those apply to throughput histories rather than to delivery-time histograms, and they are configured constants rather than reported counts. It would be wrong to say the benchmark discards nothing; what it does not do is count the latency samples its own resolution makes unusable.

The comparison is therefore not that standards bodies have failed to think about this. One of them states our rule in almost our words, for a different domain. The rule simply has not reached the place where brokers are compared on sub-millisecond paths.

39 S41. The classical statistics of a discarded zero

The main text argues that deleting the samples which compute to zero conditions the reported distribution on being positive, and that the retained fraction is therefore part of the measurement. Two classical sources sit close enough to that argument to be named, and both need care in the naming, so we give them the space here rather than compress them into a clause.

The baseline the guard invalidates. The GUM gives the standard model for a quantised reading. Clause F.2.2.1, "The resolution of a digital indication", states that if the resolution is δx then the stimulus "can lie with equal probability anywhere in the interval $X - \delta x/2$ to $X + \delta x/2$" and is therefore described by "a rectangular probability distribution … of width δx with variance $u^2 = (\delta x)^2/12$" [46]. That symmetric model is the departure point for this paper: it holds for *retained* readings, and the positivity guard of Section **??** removes one side of the interval, so neither the rectangular shape nor the $(\delta x)^2/12$ variance survives the filter. Quantisation is not the problem; the asymmetry the guard introduces on top of it is. We cite the clause rather than the formula because the same $(\delta x)^2/12$ appears in F.2.2.2 for hysteresis, and the form alone does not identify the mechanism.

The oldest statement that discarding zeros is not free. Pratt [47] showed that discarding zero differences before ranking violates a natural monotonicity requirement: a sample not judged significantly positive can become significantly positive when *every* observation is decreased. He calls this an anomaly and gives a procedure that avoids it — rank all observations including the zeros, drop only the zeros' ranks, and refer the statistic to its null distribution conditional on the number of zeros.

Two qualifications keep the analogy honest. First, Pratt *conditions on* the zero count; he does not treat it as evidence of a shift, and he explicitly declines to claim any optimality for the procedure. Our claim about the retention rate is the weaker and more practical one — that the count must be published so a reader can see what the distribution is conditioned on — and it does not need Pratt's stronger machinery. Second, the reading that makes the parallel exact is not Pratt's own framing but one he reports from Meier in a remarks section: that a zero arises because "with only our crude measurement, we do not know" the sign a finer measurement would have resolved. That is precisely the sub-quantum sample of Section **??**, and it is a 1959 statement of it; we attribute it where it belongs rather than to Pratt's main development.